

DEPARTEMENT TECHNOLOGIES DE L'INFORMATIQUE

N° 2021/

MEMOIRE DE STAGE DE FIN D'ETUDES

POUR L'OBTENTION DU DIPLOME DE MASTERE PROFESSIONNEL
EN TECHNOLOGIES DE L'INFORMATIQUE :

GÉNIE LOGICIEL ET DÉVELOPPEMENT RAPIDE D'APPLICATIONS

-ENSEIGNEMENT A DISTANCE-

Intitulé :

Développement d'un système de collecte de
données en temps réel et de gestion des alertes
pour les machines en salle de réveil

Date de soutenance :

.....

Jury :

Président :

Rapporteur :

Encadreur : Pr. Ridha Azizi

Elaboré par :

Zaibi Racha

Intitulé du stage : Stage fin d'étude du master en GLDRA

Réalisé par les étudiants : Zaibi Racha

Organisme d'accueil : Faculté de médecine de Monastir

Mots-Clés :

Résumé :

**Nom de l'Enseignant
Encadrant :**

Pr. Ridha Azizi

Signature :

**Nom de l'Encadrant
Industriel :**

**Pr. Charffeddin
Ammeri**

**Tampon & Sign.
de l'Encadrant
Ind. :**

Table des matières

Introduction Générale	10
1 Cadre Général du Projet	11
1.1 Présentation de l'organisme d'accueil	11
1.2 Contexte et problématique	11
1.3 Présentation du projet	12
1.3.1 Objectifs	12
1.3.2 Variété des moniteurs et des signes vitaux	13
1.3.3 Impact attendu	14
1.4 Etude de l'existants	14
1.4.1 Solutions disponibles	14
1.5 Solution proposée : VitalSync	16
1.5.1 Collecte de données	17
1.5.2 Gestion des alertes	18
2 Méthodologie adoptée	19
2.1 Choix de la méthodologie	19
2.1.1 Comparaison des méthodologies courantes	19
2.1.2 Rationale détaillé pour le choix de Scrum	19
2.2 Présentation de Scrum	20
2.2.1 Artefacts Scrum et leur application	20
2.2.2 Événements Scrum et leur implémentation	21
2.3 Les rôles Scrum	22
2.4 Gestion des risques avec Scrum	23
2.5 Engagement des parties prenantes dans Scrum	24
2.6 Outils de support à Scrum	24
2.7 Représentation visuelle du processus Scrum	25

3	Étude architecturale du projet	27
3.1	Introduction	27
3.2	Architecture globale	27
3.2.1	Diagramme de classes	29
3.2.2	Diagramme de séquence	31
3.2.3	Diagramme de déploiement	31
3.2.4	Diagramme de composants	33
3.2.5	Diagramme d'activité	34
3.3	Architecture physique et fonctionnement	35
3.3.1	Composants matériels	35
3.3.2	Fonctionnement	36
3.3.3	Spécifications logicielles	36
3.3.4	Flux de communication	37
3.4	Justification de l'architecture	37
3.5	Conclusion	38
4	Sprint 0	39
4.1	Introduction	39
4.2	Capture des besoins	39
4.2.1	Identification des acteurs	39
4.2.2	Besoins fonctionnels et non fonctionnels	40
4.2.3	Diagramme des cas d'utilisation global	41
4.3	Pilotage du projet avec Scrum	41
4.3.1	Acteurs Scrum et Missions	42
4.3.2	Organisation du Backlog	42
4.3.3	Planification des sprints	43
4.4	Environnement de travail	45
4.4.1	Environnement matériel	45
4.4.2	Environnement de développement	45
4.4.3	Environnement logiciel	46

4.5	Conclusion	48
5	Sprint 1 : Création du dataset et mise en place du pipeline OCR	49
5.1	Introduction	49
5.2	Objectifs du Sprint 1	49
5.2.1	Sprint Backlog	50
5.3	Tâches réalisées	50
5.3.1	Configuration du matériel	51
5.3.2	Capture et annotation des images	51
5.3.3	Entraînement du modèle de détection	51
5.3.4	Développement du pipeline OCR	52
5.3.5	Tests d'intégration et déploiement initial	52
5.4	Livrables du Sprint 1	53
5.5	Tests et validation	53
5.6	Défis et solutions	53
5.7	Rétrospective du Sprint 1	54
5.8	Planification pour le Sprint 2	54
5.9	Conclusion	54
6	Sprint 2	56
6.1	Introduction	56
6.2	Objectifs	56
6.3	Activités	56
6.3.1	Évaluation des règles d'alerte	57
6.3.2	Génération et stockage des alertes	57
6.3.3	Livraison des notifications	57
6.3.4	Confirmation des alertes	57
6.3.5	Intégration de l'API et du tableau de bord	57
6.3.6	Mise en œuvre des alertes et notifications	58
6.4	Résultats	58

6.5	Tests	58
6.6	Défis	59
6.7	Rétrospective	59
6.8	Planification du Sprint 3	59
6.9	Conclusion	60
Bibliographie		61

Table des figures

1.1	Logo de la FMM	11
1.2	Logo du SmartLab	11
1.3	Flux simplifié de VitalSync.	17
2.1	Processus Scrum.	21
2.2	Rôles Scrum.	23
2.3	Logo de ClickUp.	24
2.4	Logo de Slack.	25
2.5	Logo de Git.	25
2.6	Processus Scrum adapté au projet.	26
3.1	Architecture globale	28
3.2	Diagramme de classes	30
3.3	Diagramme de séquence global	31
3.4	Diagramme de déploiement	33
3.5	Diagramme de composants du système	34
3.6	Diagramme d'activité du flux de données	35
3.7	Schéma général du flux de communication	37
4.1	Diagramme des cas d'utilisation global	41
4.2	Logos des technologies backend : Laravel et PHP	45
4.3	Logos des technologies utilisées pour le traitement et l'API Python : Python et Flask	45
4.4	Logos des technologies utilisées pour le développement mobile : Flutter et Dart	46
4.5	Logos des technologies utilisées pour le développement web : React et JavaScript	46
4.6	Logos des outils de gestion de base de données : MySQL et phpMyAdmin	46
4.7	Logo de Visual Studio Code	47
4.8	Logo d'Android Studio	47

4.9	Logo de LaTeX	47
4.10	Logo de GitHub	47
4.11	Logo de Docker Desktop	48
5.1	Courbe d'apprentissage du modèle de détection	52
5.2	Courbe d'apprentissage du modèle de détection	52
6.1	Diagramme de séquence de la génération d'alertes et de la notification . . .	58

Liste des tableaux

1.1	Domaines couverts par le projet	13
1.2	Plages normales des signes vitaux selon l'âge et le contexte	14
1.3	Comparaison des solutions de surveillance	16
2.1	Comparaison des méthodologies de développement logiciel	19
4.1	Acteurs du système et leurs rôles	40
4.2	Acteurs du projet Scrum	42
4.3	Backlog du Projet avec Priorités MoSCoW	43
5.1	Sprint Backlog du Sprint 1	50
5.2	Précision par classe pour le modèle de détection sur le jeu de validation . .	51
5.3	Résultats des tests du Sprint 1	53

Introduction Générale

Dans un contexte médical en constante évolution, l'optimisation de la surveillance des patients en salle de réveil constitue un enjeu crucial pour assurer des soins de qualité et réactifs. En Tunisie, de nombreux établissements de santé dépendent encore de méthodes manuelles et de systèmes centralisés limités, ce qui entraîne des délais dans la détection des situations critiques. Par exemple, les infirmiers doivent vérifier physiquement les constantes vitales des patients, ce qui augmente le risque d'erreurs humaines et la charge de travail, particulièrement dans les hôpitaux à ressources limitées où les équipements modernes sont rares.

Ce projet, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, s'inscrit dans une démarche d'innovation visant à moderniser la surveillance post-opératoire. Il propose un système de collecte de données en temps réel et de gestion des alertes, combinant l'extraction de données via la reconnaissance optique de caractères (OCR) pour les moniteurs. L'objectif principal est d'améliorer la réactivité du personnel médical face aux situations critiques, tout en réduisant les interventions manuelles et les erreurs.

Ce système répond à plusieurs besoins critiques dans le domaine médical :

- **Pour les patients :** Une détection rapide des anomalies vitales (fréquence cardiaque, pression artérielle, saturation en oxygène) permet une intervention immédiate, améliorant les résultats cliniques.
- **Pour le personnel médical :** Les notifications en temps réel via mobile et web réduisent la nécessité de déplacements constants, permettant aux infirmiers et médecins de se concentrer sur les cas prioritaires.
- **Pour les hôpitaux :** L'automatisation de la surveillance améliore l'efficacité opérationnelle et réduit les coûts liés aux erreurs humaines ou aux retards d'intervention.

En outre, la solution est conçue pour être évolutive, adaptable à différents types de moniteurs et conforme aux réglementations médicales en matière de sécurité des données.

Chapitre 1

Cadre Général du Projet

1.1 Présentation de l'organisme d'accueil

Le projet a été réalisé au sein du **SmartLab**, une **Unité de Service Commun et de Recherche (USCR)** rattachée à la **Faculté de Médecine de Monastir (FMM)**, fondée le 15 août 1980 [1]. La FMM est une institution tunisienne de référence, reconnue pour son excellence académique et son engagement dans la recherche médicale. Elle a obtenu une accréditation internationale et la certification **ISO 21001**, attestant de ses standards élevés en éducation et recherche [2]. La FMM forme des professionnels de santé qualifiés et contribue à l'avancement des sciences médicales à travers des projets innovants et des partenariats interdisciplinaires.

Le SmartLab, créé pour catalyser l'innovation dans les sciences de la santé, joue un rôle clé dans la transformation des soins médicaux en Tunisie [3]. En favorisant la collaboration entre chercheurs, cliniciens et start-ups, il développe des solutions technologiques adaptées aux défis locaux, tels que les contraintes budgétaires et les besoins en automatisation. Le SmartLab dispose d'équipements de pointe, comme des laboratoires IoT et des outils de simulation, et d'une équipe pluridisciplinaire, ce qui en fait une plateforme idéale pour des projets comme VitalSync, qui vise à moderniser la surveillance post-opératoire. Cette unité s'inscrit dans une vision stratégique visant à combler le fossé entre la recherche académique et les applications cliniques, contribuant à l'amélioration des soins de santé en Tunisie et au-delà.



FIGURE 1.1 : Logo de la FMM



FIGURE 1.2 : Logo du SmartLab

1.2 Contexte et problématique

En Tunisie, les hôpitaux publics et privés opèrent dans un contexte marqué par des contraintes financières et logistiques significatives [4]. La surveillance des patients en salle de réveil

repose sur des méthodes traditionnelles : un écran centralisé, situé dans une salle administrative, affiche les signes vitaux, tandis que le personnel médical effectue des vérifications manuelles régulières. Ce système entraîne des inefficacités majeures. Par exemple, dans un hôpital moyen tunisien, un infirmier peut être responsable de 10 à 15 patients en post-opératoire, nécessitant des déplacements fréquents pour vérifier les moniteurs, ce qui peut retarder la détection d'anomalies critiques, comme une chute de SpO_2 , de plusieurs minutes [5]. Ces délais augmentent les risques pour les patients, particulièrement dans des situations nécessitant une intervention immédiate.

Le manque de personnel qualifié aggrave ces défis, avec un ratio infirmier-patient souvent inférieur aux normes internationales (e.g., 1 :12 contre 1 :6 recommandé par l'OMS dans les unités post-opératoires) [6]. Cette pénurie exacerbe la charge de travail et les risques d'erreurs humaines, notamment en période d'affluence. Les solutions existantes, principalement étrangères, telles que Mindray, ChartX ou IQ Messenger SmartApp, sont peu adaptées. Leur coût d'acquisition, souvent supérieur à 50 000–100 000 dinars tunisiens pour un hôpital moyen, and leurs besoins en infrastructure réseau sophistiquée ou en matériel spécifique les rendent inaccessibles [7, 8]. De plus, ces solutions sont rarement compatibles avec les équipements hétérogènes des hôpitaux tunisiens, qui utilisent un mélange de moniteurs anciens et modernes de marques variées [6]. Cette problématique souligne la nécessité d'une solution locale, économique et flexible, capable d'automatiser la surveillance tout en s'intégrant aux infrastructures existantes.

1.3 Présentation du projet

Le projet **VitalSync** ambitionne de révolutionner la surveillance post-opératoire dans les salles de réveil tunisiennes en développant un système de collecte de données en temps réel et de gestion des alertes. En exploitant des technologies modernes comme l'**Internet des Objets (IoT)** [9], la **reconnaissance optique de caractères (OCR)** [10] et les **notifications en temps réel** [11], VitalSync répond aux défis d'automatisation et de réactivité dans des environnements à ressources limitées. Conçu pour être économique, il utilise des caméras IP et un serveur local, assurant une compatibilité avec une large gamme de moniteurs médicaux, qu'ils soient anciens ou modernes, contrairement aux solutions étrangères coûteuses.

1.3.1 Objectifs

VitalSync vise à automatiser la collecte des paramètres vitaux, tels que la fréquence cardiaque, la pression artérielle et la SpO_2 , en utilisant l'OCR pour les moniteurs (e.g., Mindray)

[10]. Le système permet une gestion dynamique des alertes basées sur des seuils critiques personnalisables, adaptés aux profils des patients (e.g., adultes, enfants, patients chroniques). Les notifications sont envoyées via Firebase Cloud Messaging (FCM) [11], garantissant une réactivité optimale. VitalSync ambitionne également de réduire les vérifications manuelles, de minimiser les délais d’alerte et les erreurs humaines, améliorant la sécurité des patients et l’efficacité du personnel médical.

Le projet couvre l’ensemble de la chaîne de traitement des données, comme illustré dans le Tableau 1.1. L’acquisition repose sur l’extraction des données via OCR pour les moniteurs, avec une validation automatique pour garantir la fiabilité [10]. Le traitement inclut le nettoyage des données, leur agrégation et l’analyse des tendances des signes vitaux, permettant de détecter des anomalies subtiles. La visualisation est assurée par une interface utilisateur intuitive, offrant des tableaux de bord en temps réel et des historiques des données, accessibles sur des appareils fixes ou mobiles.

TABLE 1.1 : Domaines couverts par le projet

Phase	Description
Acquisition	Extraction des données via OCR
Traitement	Nettoyage, agrégation et analyse des tendances des signes vitaux
Visualisation	Interface avec tableaux de bord et historiques des données

1.3.2 Variété des moniteurs et des signes vitaux

Les hôpitaux tunisiens utilisent une grande variété de moniteurs médicaux, reflétant les contraintes budgétaires et l’évolution technologique inégale du secteur [12]. Les établissements combinent des moniteurs anciens, comme les modèles analogiques de Philips ou B. Braun, avec des équipements modernes, tels que les moniteurs Mindray, Edan ou HealForce équipés d’interfaces numériques. Ces moniteurs varient par leur format d’affichage (e.g., LED vs. CRT), leurs protocoles de communication (e.g., absence de connectivité vs. API privée) et leur ancienneté. Par exemple, un moniteur ancien peut afficher la fréquence cardiaque en texte brut sans sortie numérique, nécessitant l’OCR [10], tandis qu’un moniteur Mindray moderne transmet les données via un protocole HL7 [7]. Cette hétérogénéité exige une solution flexible, comme VitalSync, capable de s’adapter à divers équipements.

Les signes vitaux surveillés, notamment la fréquence cardiaque, la pression artérielle et la SpO₂, présentent une variabilité significative selon l’âge, l’état de santé et le contexte de soins [13]. Le Tableau 1.2 illustre les plages normales pour différents groupes. Par exemple, la fréquence cardiaque d’un adulte au repos (60–100 bpm) est plus faible que celle d’un enfant

(70–120 bpm), tandis qu’un patient post-opératoire peut présenter des fluctuations indiquant des complications. La pression artérielle varie chez les patients hypertendus (>140/90 mmHg) ou hypotendus, et la SpO₂ peut chuter chez ceux atteints de pathologies respiratoires (<90% en cas de pneumonie). En salle de réveil, une SpO₂ inférieure à 90% ou une fréquence cardiaque supérieure à 120 bpm peut signaler une urgence, mais ces seuils doivent être ajustés selon le patient. VitalSync intègre des seuils d’alerte dynamiques, configurables par le personnel médical, et assure une compatibilité multi-marques pour gérer cette diversité.

TABLE 1.2 : Plages normales des signes vitaux selon l’âge et le contexte

Signe vital	Adulte (repos)	Enfant (3–12 ans)	Post-opératoire (adulte)
Fréquence cardiaque (bpm)	60–100	70–120	70–120
Pression artérielle (mmHg)	90/60–120/80	80/50–110/70	100/60–140/90
SpO ₂ (%)	95–100	95–100	92–100

1.3.3 Impact attendu

L’implémentation de VitalSync devrait transformer les soins post-opératoires en Tunisie. Sur le plan clinique, la détection précoce des anomalies grâce à des alertes en temps réel permettrait de réduire significativement les complications post-opératoires observées dans les salles de réveil tunisiennes [5]. Une intervention rapide à la suite d’une chute de la saturation en oxygène peut prévenir des hypoxémies graves. Sur le plan opérationnel, la diminution notable des vérifications manuelles libérerait les infirmiers de certaines tâches répétitives, leur permettant de se concentrer davantage sur les soins directs aux patients, ce qui améliorerait l’efficacité dans un contexte de pénurie de personnel soignant [6]. D’un point de vue économique, l’utilisation d’une infrastructure simple et accessible, comme des caméras connectées à un serveur local, rend VitalSync particulièrement adapté aux ressources disponibles localement [3]. Enfin, ce projet pourrait ouvrir la voie à d’autres innovations en santé connectée, stimulant l’écosystème technologique national.

1.4 Etude de l’existants

1.4.1 Solutions disponibles

Trois solutions étrangères dominent actuellement le marché : le système de surveillance centralisé de Mindray, la plateforme web ChartX et l’application IQ Messenger SmartApp. Mindray, très répandu, repose sur un écran centralisé pour afficher les signes vitaux de plusieurs patients. Bien qu’il soit reconnu pour sa robustesse, il est limité à ses propres

équipements, excluant ainsi des marques pourtant courantes en Tunisie, comme Edan ou HealForce [7]. De plus, l'absence de notifications mobiles impose au personnel de rester à proximité de l'écran, ce qui peut nuire à la réactivité en cas d'urgence. Enfin, son coût élevé, tant à l'acquisition qu'à l'entretien, en fait une solution difficilement accessible pour de nombreux établissements de santé locaux [8].

ChartX, une plateforme web, permet une automatisation partielle en collectant des données provenant de divers équipements médicaux via du matériel intermédiaire, tel que des passerelles IoT. Elle propose des tableaux de bord ainsi que des alertes web. Toutefois, son bon fonctionnement dépend d'une connectivité réseau fiable, souvent défaillante dans les hôpitaux situés en zones rurales en Tunisie [8]. Sa mise en place nécessite des ressources techniques importantes, notamment des serveurs dédiés et une configuration complexe, ce qui complique son adoption.

IQ Messenger SmartApp, conçue pour être indépendante des fabricants, se distingue par sa capacité à générer des alertes en temps réel sur smartphones via des réseaux sans fil, sans recourir à des services cloud [14]. Elle offre une intégration avec les dossiers médicaux informatisés et inclut un mode « occupé » pour limiter la surcharge d'alertes. Toutefois, sa compatibilité reste restreinte à certains fabricants spécifiques, excluant une partie importante des équipements utilisés dans les établissements tunisiens. De plus, son coût demeure un frein pour les structures disposant de budgets limités [8].

Critiques des solutions existantes

Les solutions existantes pour la surveillance en salle de réveil présentent des limites significatives dans le contexte tunisien. Mindray, bien qu'efficace pour une surveillance centralisée, manque d'automatisation avancée et de réactivité. L'absence d'alertes mobiles oblige le personnel à surveiller constamment un écran centralisé, ce qui entraîne des délais, notamment en période de forte affluence (e.g., 15 patients par infirmier). Comme mentionné dans [7], Mindray est également limité à ses propres appareils, réduisant sa flexibilité dans des environnements hétérogènes.

ChartX, bien que compatible avec plusieurs marques, nécessite une infrastructure réseau sophistiquée, un obstacle majeur dans les hôpitaux tunisiens où la connectivité est souvent instable. Une panne réseau peut désactiver les alertes, compromettant la sécurité des patients, comme noté dans [15]. IQ Messenger SmartApp offre des alertes rapides et une intégration avec les dossiers hospitaliers électroniques (DHE), mais sa compatibilité est restreinte à certains fabricants (e.g., Philips, B. Braun), ce qui pose problème dans des environnements multi-marques. De plus, le coût élevé de ces solutions et leur dépendance à des infrastructures complexes limitent leur adoption dans les hôpitaux tunisiens aux budgets contraints [8].

Le Tableau 1.3 présente une comparaison des solutions de surveillance en salle de réveil, mettant en évidence leurs forces et faiblesses. Les critères incluent l'automatisation avancée, la compatibilité des appareils, les alertes en temps réel, la simplicité matérielle et l'efficacité opérationnelle. VitalSync se distingue par sa capacité à automatiser la collecte des données et à s'adapter à une large gamme de moniteurs, tout en offrant des alertes instantanées sur les appareils mobiles du personnel soignant.

TABLE 1.3 : Comparaison des solutions de surveillance

Critère	Mindray	ChartX	IQ Messenger
Automatisation avancée	∅	±	✓
Compatibilité appareils	∅	✓	±
Alertes en temps réel	∅	±	✓
Simplicité matérielle	±	∅	✓
Efficacité opérationnelle	±	±	✓

Légende :

- ✓ : Solution optimale
- ± : Solution moyenne
- ∅ : Solution non adaptée

1.5 Solution proposée : VitalSync

VitalSync constitue une solution innovante spécifiquement conçue pour répondre aux besoins des hôpitaux tunisiens, comme illustré dans la Figure 1.3. En s'appuyant sur les technologies IoT [16], l'OCR (à l'aide d'OpenCV et Tesseract) [10] et les API, le système collecte automatiquement les paramètres vitaux à partir de moniteurs, assurant une compatibilité étendue avec diverses marques présentes sur le marché local.

VitalSync permet une gestion dynamique des alertes en fonction de seuils critiques configurables, et envoi des notifications instantanées via Firebase Cloud Messaging [11]. Grâce à l'utilisation de caméras IP et d'un serveur local, la solution évite les besoins en infrastructure complexe ou coûteuse. Elle vise à alléger la charge de travail des soignants en automatisant une grande partie des vérifications, tout en assurant une meilleure réactivité dans la prise en charge des anomalies.

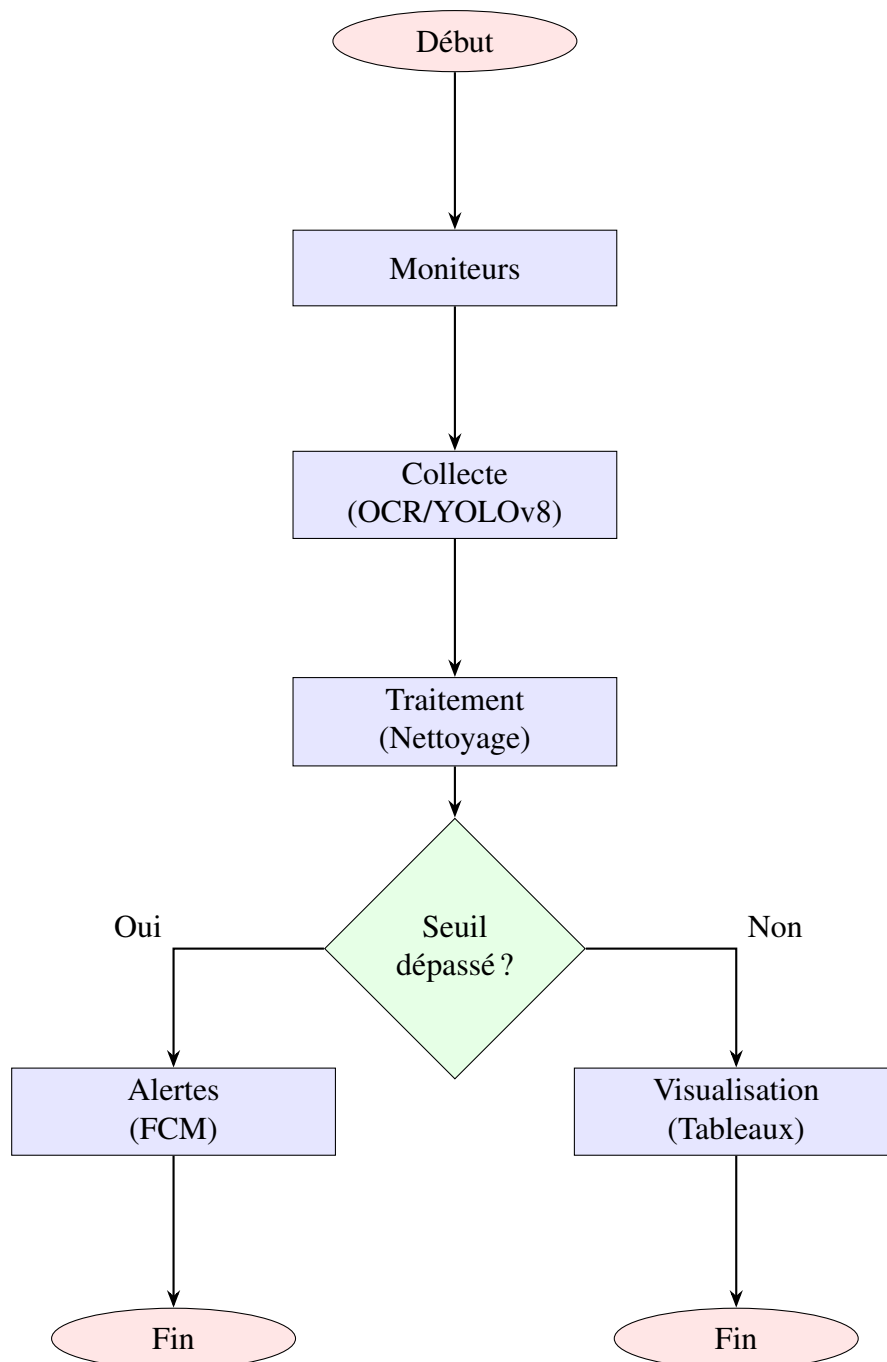


FIGURE 1.3 : Flux simplifié de VitalSync.

1.5.1 Collecte de données

VitalSync exploite des caméras IP couplées à **OpenCV** et **Tesseract OCR** pour extraire les données de moniteurs analogiques encore largement utilisés dans le contexte tunisien [10]. Pour les équipements plus récents, l'intégration via des API standards permet une collecte fiable et en temps réel des paramètres vitaux [16]. Le système applique automatiquement des règles de validation pour éliminer les erreurs dues à des conditions d'éclairage médiocres

ou à des perturbations visuelles, garantissant la fiabilité des informations traitées. À titre d'illustration, l'OCR permet d'interpréter des lectures affichées sur des écrans CRT, tandis que les API assurent une extraction directe et structurée des données issues de moniteurs numériques.

1.5.2 Gestion des alertes

Le système repose sur une logique de seuils configurables en fonction du profil et de l'état clinique des patients. Les notifications sont transmises via **Firestore Cloud Messaging** sur les appareils mobiles des soignants ainsi que par courriel, afin de favoriser une intervention rapide [11]. Le dispositif permet également une gestion intelligente des priorités : les alertes critiques sont mises en avant et redistribuées automatiquement si elles ne sont pas traitées dans un délai raisonnable, garantissant une continuité de surveillance même en cas d'indisponibilité momentanée du personnel.

Chapitre 2

Méthodologie adoptée

2.1 Choix de la méthodologie

2.1.1 Comparaison des méthodologies courantes

Le tableau suivant (Tableau 2.1) compare les méthodologies les plus couramment utilisées dans le développement logiciel, en mettant en évidence leurs caractéristiques clés :

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d'implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Scalabilité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

TABLE 2.1 : Comparaison des méthodologies de développement logiciel

2.1.2 Rationale détaillé pour le choix de Scrum

Le choix de la méthodologie Scrum pour ce projet est motivé par plusieurs facteurs spécifiques au contexte du développement d'un système de collecte de données en temps réel et de gestion des alertes dans un environnement médical. Premièrement, l'approche itérative de Scrum permet de recueillir des retours continus des parties prenantes, notamment le personnel médical du SmartLab de la Faculté de Médecine de Monastir, pour affiner les exigences fonctionnelles et non fonctionnelles. Cela est particulièrement crucial dans un projet médical où la précision et la réactivité sont essentielles pour garantir la sécurité des patients.

Deuxièmement, Scrum offre une grande adaptabilité face aux incertitudes techniques, telles que l'extraction de données à partir de moniteurs via OCR. Ces sources de données présentent

des défis distincts, notamment en termes de précision de l'OCR. Scrum permet de tester et valider ces composants dès les premiers sprints, réduisant ainsi les risques d'échec technique. Enfin, la méthodologie Scrum favorise une collaboration étroite entre les développeurs, le Product Owner et le Scrum Master, assurant une communication fluide avec les parties prenantes médicales. Cette collaboration est essentielle pour répondre aux exigences changeantes du domaine médical, où les besoins peuvent évoluer rapidement en fonction des retours des utilisateurs ou des contraintes réglementaires [17, 18].

2.2 Présentation de Scrum

Comme présenté dans la figure 2.1, Scrum est un framework agile qui repose sur des cycles de développement appelés sprints. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés [19] :

- **Product Backlog** : Liste des fonctionnalités à développer, priorisée par le Product Owner en fonction des besoins des utilisateurs et des objectifs du projet.
- **Sprint Backlog** : Ensemble des tâches à réaliser pendant un sprint, défini par l'équipe de développement.
- **Sprint Planning** : Réunion permettant de définir les objectifs et les tâches du sprint.
- **Daily Scrum** : Brève réunion quotidienne de l'équipe pour suivre l'avancement et identifier les obstacles.
- **Sprint Review** : Présentation de l'incrément produit aux parties prenantes à la fin de chaque sprint.
- **Sprint Retrospective** : Analyse du sprint pour identifier les axes d'amélioration.

2.2.1 Artefacts Scrum et leur application

Scrum repose sur trois artefacts principaux qui structurent le développement du projet : le *Product Backlog*, le *Sprint Backlog* et l'*Incrément*. Ces artefacts sont appliqués comme suit dans le cadre du projet [20] :

- **Product Backlog** : Le *Product Backlog* regroupe l'ensemble des fonctionnalités à développer, telles que définies dans le tableau ?? (voir section ??). Il est géré par le Product Owner, qui priorise les tâches en utilisant la méthode MoSCoW pour classer les fonctionnalités en *Must*, *Should*, *Could* et *Won't*. Ce backlog est régulièrement affiné pour intégrer les retours des parties prenantes, notamment le personnel médical,

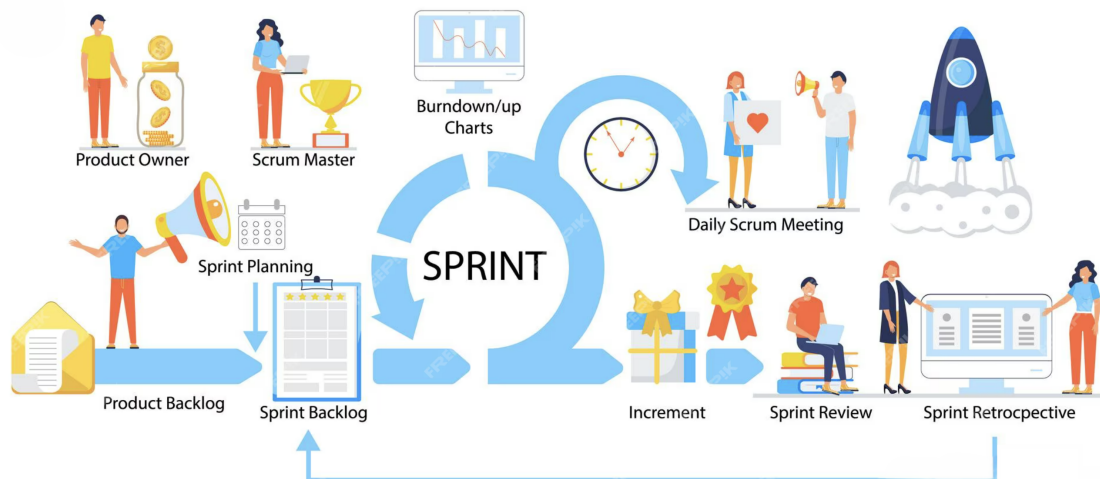


FIGURE 2.1 : Processus Scrum.

afin de garantir que les fonctionnalités prioritaires répondent aux besoins critiques du système.

- **Sprint Backlog :** À chaque sprint, un sous-ensemble de tâches est sélectionné à partir du *Product Backlog* pour constituer le *Sprint Backlog*. Par exemple, pour le Sprint 1, les tâches se concentrent sur l'intégration de la collecte de données basée sur caméra et l'extraction OCR (voir section ??). Ces tâches sont définies lors de la réunion de planification du sprint, en collaboration avec l'équipe de développement.
- **Incrément :** Chaque sprint produit un incrément fonctionnel et potentiellement livrable. Par exemple, à l'issue du Sprint 1, un pipeline OCR fonctionnel est livré, permettant la capture et l'extraction des données vitales à partir des moniteurs. Cet incrément peut être testé par le personnel médical pour valider sa précision et sa fiabilité avant d'intégrer de nouvelles fonctionnalités dans les sprints suivants.

2.2.2 Événements Scrum et leur implémentation

Scrum repose sur des événements clés qui structurent le processus de développement. Ces événements sont implémentés comme suit dans le cadre du projet [17] :

- **Planification de sprint :** Lors de la réunion de planification, l'équipe sélectionne les tâches du *Product Backlog* à inclure dans le *Sprint Backlog*, en estimant l'effort requis (voir tableau ?? pour le Sprint 1). Par exemple, pour le Sprint 1, l'intégration de l'OCR a été estimée à 5 jours, en tenant compte de la configuration des caméras et du pipeline de traitement d'images.
- **Daily Scrum :** Des réunions quotidiennes de 15 minutes sont organisées pour suivre l'avancement des tâches et identifier les obstacles. Par exemple, si des problèmes de

qualité d'image affectent l'OCR, ils sont discutés et résolus rapidement, souvent en ajustant les paramètres de prétraitement d'OpenCV.

- **Revue de sprint** : À la fin de chaque sprint, l'incrément est présenté aux parties prenantes, comme le pipeline OCR fonctionnel à la fin du Sprint 1. Les médecins et infirmiers testent l'incrément pour valider sa précision et son utilité clinique, fournissant des retours pour le sprint suivant.
- **Rétrospective de sprint** : Après chaque sprint, l'équipe analyse les succès et les défis rencontrés. Par exemple, si des erreurs d'OCR sont détectées lors du Sprint 1, la rétrospective peut conduire à l'ajout de tests supplémentaires ou à l'amélioration du prétraitement des images dans le Sprint 2.

2.3 Les rôles Scrum

Dans Scrum, trois rôles principaux sont définies pour assurer le bon fonctionnement du processus de développement [21] :

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

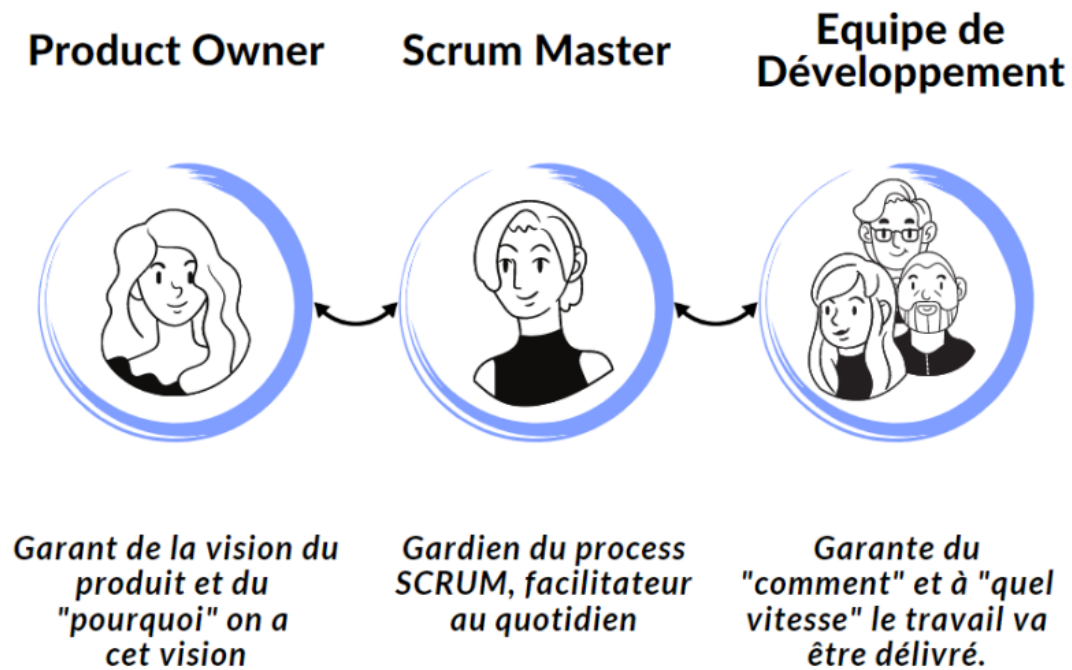


FIGURE 2.2 : Rôles Scrum.

2.4 Gestion des risques avec Scrum

Le développement d'un système de collecte de données en temps réel et de gestion des alertes dans un contexte médical implique plusieurs risques, notamment techniques et opérationnels. Scrum offre un cadre efficace pour identifier, gérer et atténuer ces risques grâce à son approche itérative et ses mécanismes de revue réguliers [22].

- **Risques techniques** : L'extraction des données via OCR peut souffrir d'une précision insuffisante en raison de la qualité des images capturées ou de la variabilité des écrans des moniteurs. Ces risques sont atténués par des tests précoces dans les sprints initiaux, comme le Sprint 1, qui se concentre sur la validation de la collecte de données .
- **Risques opérationnels** : Les retards dans la livraison des alertes en temps réel peuvent compromettre la réactivité du personnel médical. Scrum permet de tester les performances des notifications dès le Sprint 2, en intégrant Firebase Cloud Messaging (FCM) et en validant les temps de réponse .
- **Revue et amélioration** : Les rétrospectives de sprint permettent d'identifier les problèmes liés aux processus, comme les goulots d'étranglement dans la communication entre l'équipe de développement et le personnel médical. Par exemple, si des retours indiquent des besoins supplémentaires pour l'interface utilisateur, ceux-ci peuvent être intégrés dans le *Product Backlog* pour le sprint suivant.

2.5 Engagement des parties prenantes dans Scrum

L'implication des parties prenantes est un pilier central de la méthodologie Scrum, particulièrement dans un projet médical où les besoins des utilisateurs finaux (médecins, infirmiers, administrateurs) sont critiques [17, 18]. Dans ce projet, les parties prenantes, notamment le personnel médical du SmartLab et les chercheurs de la Faculté de Médecine de Monastir, jouent un rôle actif à plusieurs niveaux :

- **Revue de sprint** : À la fin de chaque sprint, les parties prenantes participent aux réunions de revue pour évaluer les incréments livrés, tels que le pipeline OCR du Sprint 1 ou le système de notification du Sprint 2. Leurs retours permettent de valider la conformité des fonctionnalités aux besoins cliniques, par exemple en vérifiant la précision des données extraites ou l'ergonomie de l'interface utilisateur.
- **Affinement du Product Backlog** : Le Product Owner collabore avec les parties prenantes pour prioriser les fonctionnalités, en utilisant la méthode MoSCoW pour classer les tâches (voir tableau 4.3) [19]. Par exemple, les médecins peuvent insister sur la priorisation des alertes critiques (M-02) pour garantir une réactivité maximale.
- **Conformité réglementaire** : Les parties prenantes, en particulier les experts médicaux, contribuent à garantir que le système respecte les réglementations médicales, comme la protection des données des patients via le chiffrement (M-07). Leur expertise est essentielle pour aligner le développement sur les normes éthiques et légales.

2.6 Outils de support à Scrum

Pour faciliter la mise en œuvre de Scrum, plusieurs outils numériques sont utilisés pour gérer les processus, la communication et le suivi des tâches :

- **ClickUp** : Utilisé pour gérer le *Product Backlog* et le *Sprint Backlog*, ClickUp permet de suivre les tâches, d'assigner des responsabilités et de visualiser l'avancement des sprints à l'aide de tableaux Kanban ou Scrum [23]. Par exemple, les tâches du Sprint 1, comme l'intégration de l'OCR (M-02), sont suivies via des tickets ClickUp avec des estimations de temps et des statuts mis à jour quotidiennement.



FIGURE 2.3 : Logo de ClickUp.

- **Slack** : Cette plateforme de communication permet des échanges en temps réel entre l'équipe de développement, le Product Owner et le Scrum Master [24]. Les *Daily Scrum* peuvent être partiellement gérés via des canaux dédiés, où les membres signalent leurs progrès et obstacles.



FIGURE 2.4 : Logo de Slack.

- **Git** : Utilisé pour le contrôle de version, Git permet de gérer le code source du projet et de faciliter la collaboration entre les développeurs [25]. Les commits sont alignés avec les tâches du *Sprint Backlog*, garantissant une traçabilité des contributions.



FIGURE 2.5 : Logo de Git.

2.7 Représentation visuelle du processus Scrum

La figure 2.6 illustre le processus Scrum adapté au projet, montrant la séquence des événements et les boucles de rétroaction. Chaque sprint commence par une planification, suivie de réunions quotidiennes (*Daily Scrum*), d'une revue de sprint pour présenter l'incrément, et d'une rétrospective pour améliorer le processus. Ce cycle itératif garantit que les fonctionnalités, comme la collecte de données via OCR ou la gestion des alertes, sont développées et validées progressivement.

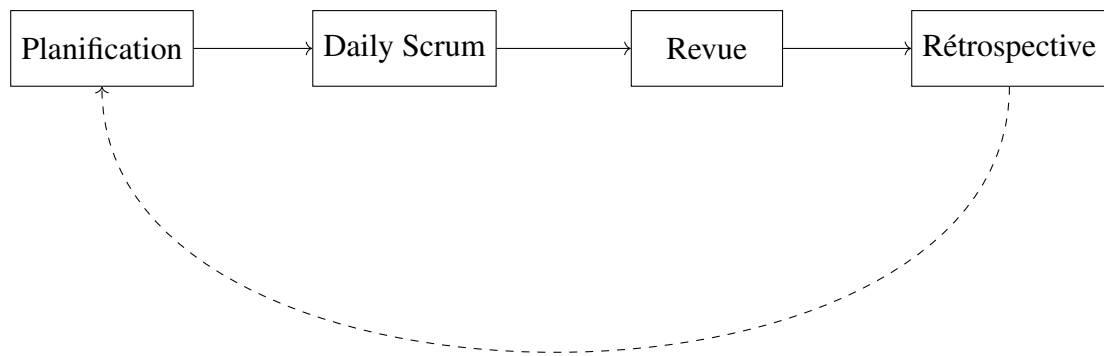


FIGURE 2.6 : Processus Scrum adapté au projet.

Chapitre 3

Étude architecturale du projet

3.1 Introduction

Ce chapitre présente l'architecture globale du système de collecte de données en temps réel et de gestion des alertes pour les machines en salle de réveil. L'objectif est de décrire les choix technologiques, les composants clés et les interactions entre les différents modules pour répondre aux besoins fonctionnels et non fonctionnels identifiés.

Ce système est conçu pour répondre à un contexte hospitalier sensible, exigeant une haute disponibilité, une sécurité accrue des données, et une capacité d'adaptation à différents environnements techniques.

3.2 Architecture globale

Le système VitalSync repose sur une architecture modulaire et évolutive, conçue pour collecter, traiter, stocker et visualiser les données vitales des patients en temps réel. Cette architecture est organisée en trois couches principales, chacune jouant un rôle spécifique dans le traitement des données médicales, comme illustré dans la Figure 3.1.

- **Couche de collecte des données** : Cette couche est responsable de l'acquisition des données brutes à partir des équipements médicaux. Elle intègre des moniteurs médicaux équipés de caméras IP qui capturent les images des écrans affichant les signes vitaux, tels que la fréquence cardiaque ou la saturation en oxygène (SpO2).
- **Couche de traitement et stockage** : Cette couche centralise le traitement des données collectées et leur stockage. Les images capturées sont prétraitées avec OpenCV pour améliorer leur qualité, suivies d'une détection des zones d'intérêt (par exemple, les valeurs des signes vitaux) à l'aide de l'algorithme YOLO. Tesseract OCR extrait ensuite les valeurs numériques ou textuelles des images. Un service Flask gère le traitement des images et communique avec un backend Laravel, qui orchestre la gestion des utilisateurs, des données des patients et des alertes. Une base de données MySQL stocke les informations des patients, les mesures vitales et l'historique des alertes pour un accès rapide et sécurisé.

- **Couche de visualisation et notification** : Cette couche permet aux utilisateurs d'interagir avec le système via des interfaces utilisateur modernes. Une application web développée avec React et une application mobile développée avec Flutter offrent un accès aux données des patients et à la gestion des alertes via une API REST. Les notifications en temps réel sont envoyées via Firebase Cloud Messaging (FCM) pour les alertes push sur les appareils mobiles, tandis que les alertes critiques sont transmises par e-mail pour garantir une communication rapide avec le personnel médical.

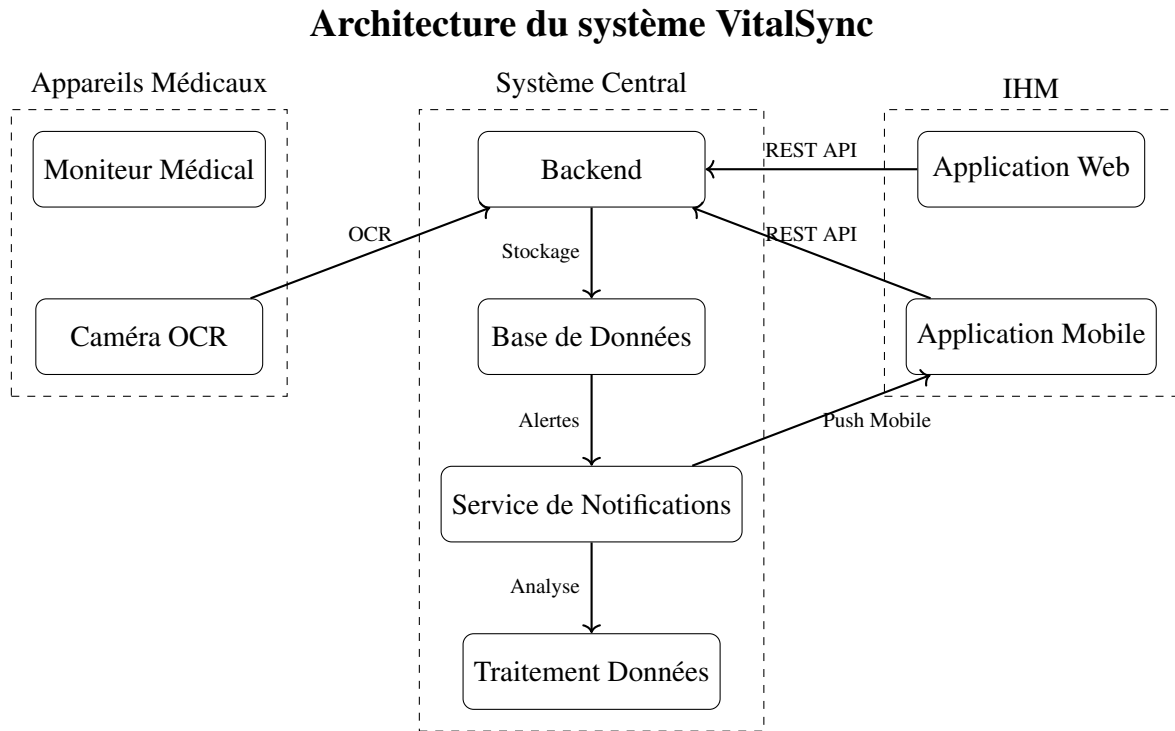


FIGURE 3.1 : Architecture globale

La Figure 3.1 présente une vue d'ensemble de l'architecture du système VitalSync, mettant en évidence les interactions entre les différentes couches. Le bloc "Appareils Médicaux" regroupe les moniteurs médicaux et les caméras OCR, qui capturent les données brutes sous forme d'images. Ces images sont transmises au bloc "Système Central" via un flux OCR, où elles sont traitées par des algorithmes (OpenCV, YOLO, Tesseract) pour extraire les signes vitaux. Le backend Laravel coordonne les opérations, stocke les données dans une base MySQL et gère les alertes via un service de notifications. Les interfaces utilisateur (web et mobile) accèdent aux données via des API REST, tandis que les notifications push (FCM) et les e-mails assurent une communication rapide des alertes critiques. Cette architecture garantit une intégration fluide des composants, une scalabilité pour gérer un grand volume de données et une réactivité pour les alertes en temps réel.

3.2.1 Diagramme de classes

Le système VitalSync utilise une base de données MySQL pour stocker les données des patients, les signes vitaux et les alertes. Le diagramme de classes (Figure 3.2) illustre le schéma de la base de données, incluant les entités clés (par exemple, Patient, Mesure, Alerte) et leurs relations. Ce schéma garantit une structure relationnelle robuste pour la gestion des données. Pour les détails d'implémentation, voir le Chapitre 5.

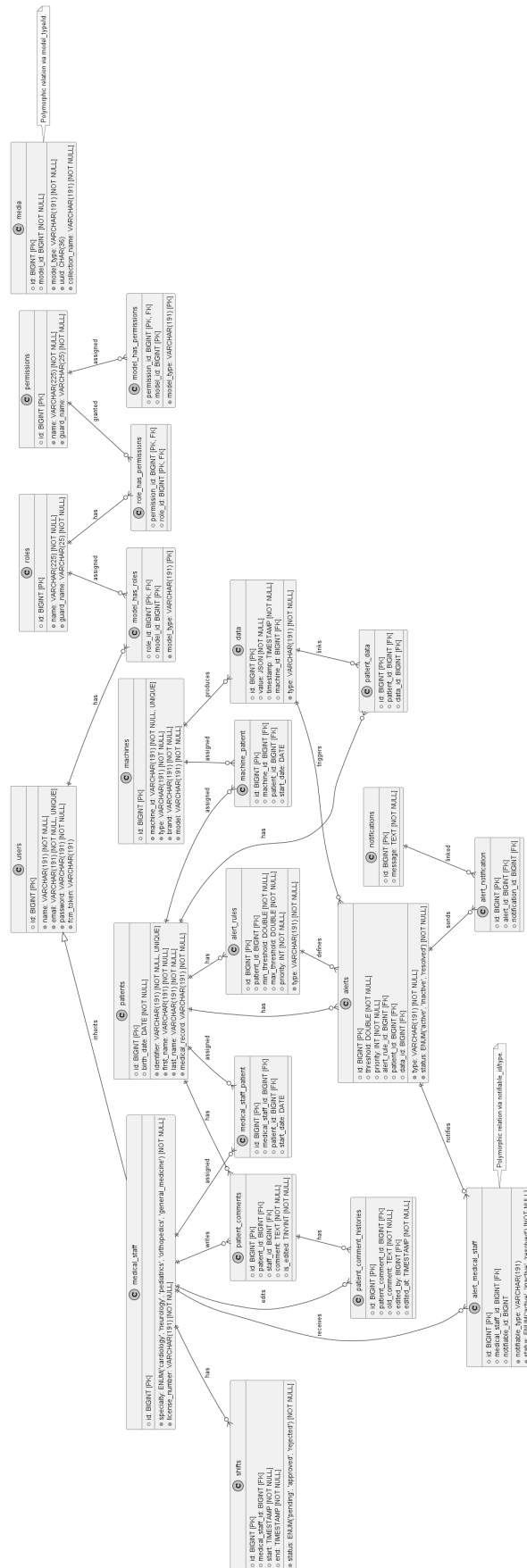


FIGURE 3.2 : Diagramme de classes

3.2.2 Diagramme de séquence

Le système VitalSync traite les signes vitaux en temps réel, depuis la capture des images par les caméras jusqu'à la notification du personnel médical via FCM. Le diagramme de séquence (Figure 3.3) illustre le workflow global, mettant en évidence les interactions entre la caméra, le traitement YOLOv8, le système d'alerte et le système de notification. Ce workflow garantit une chaîne de traitement efficace et réactive. Pour les détails d'implémentation du Sprint 1, voir le Chapitre 5.

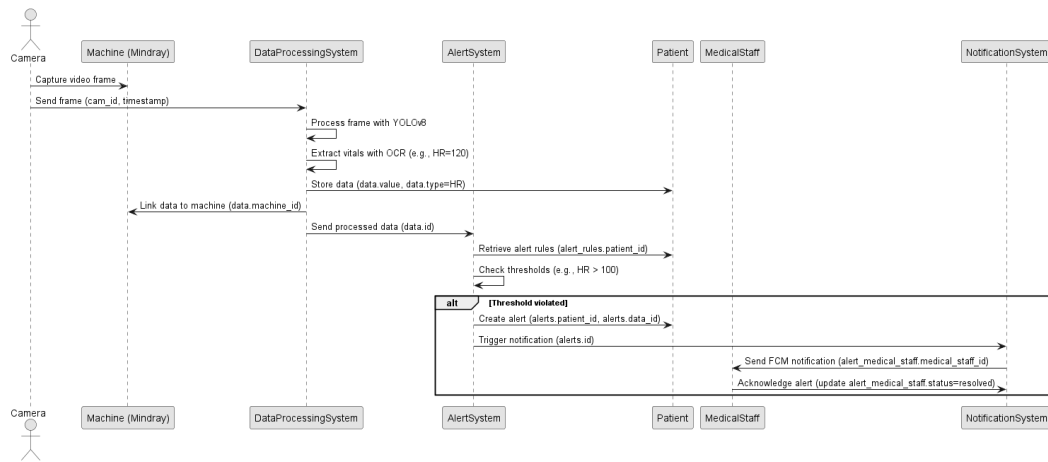


FIGURE 3.3 : Diagramme de séquence global

3.2.3 Diagramme de déploiement

Le système VitalSync s'appuie sur une architecture distribuée qui intègre des dispositifs médicaux, des serveurs applicatifs, des services de traitement d'images et des interfaces utilisateur. Cette architecture est structurée en couches distinctes, chacune hébergeant des composants matériels et logiciels spécifiques, comme illustré dans la Figure 3.4. Les sections suivantes décrivent les rôles et les interactions des composants au sein de chaque couche, ainsi que l'infrastructure réseau et les protocoles de communication qui assurent leur interconnexion.

Couche de collecte des données La couche de collecte des données est dédiée à l'acquisition des informations vitales des patients. Elle comprend des moniteurs médicaux qui mesurent en continu les signes vitaux, tels que la fréquence cardiaque ou la saturation en oxygène (SpO2). Ces moniteurs sont associés à des caméras IP qui capturent les écrans affichant ces données sous forme d'images. Les images capturées sont ensuite transmises au système central pour traitement, garantissant une collecte précise et en temps réel des données brutes.

Couche de traitement et stockage La couche de traitement et stockage constitue le cœur fonctionnel du système, hébergé sur un serveur central. Ce serveur exécute l'API Laravel pour la gestion des utilisateurs, des données des patients et des alertes, ainsi qu'un service Flask dédié au traitement des images. Le traitement des images s'appuie sur plusieurs technologies : OpenCV effectue un prétraitement pour améliorer la qualité des images, YOLO détecte les régions d'intérêt (par exemple, les zones affichant les valeurs vitales), et Tesseract OCR extrait les données textuelles ou numériques de ces régions. Une base de données MySQL stocke les informations des patients, les mesures vitales et l'historique des alertes, offrant un accès rapide et sécurisé aux données pour les analyses et les rapports.

Couche de visualisation et notification La couche de visualisation et notification permet au personnel médical d'interagir avec le système et de recevoir des alertes en temps réel. Une application mobile développée avec Flutter offre une interface pour consulter les données des patients et gérer les alertes directement sur les appareils mobiles. De même, une application web basée sur React fournit une interface utilisateur complète pour le personnel médical, accessible via des navigateurs. Les notifications en temps réel sont gérées par Firebase Cloud Messaging (FCM), qui envoie des alertes push aux appareils mobiles pour signaler des événements critiques, garantissant une réactivité immédiate.

Infrastructure et support L'infrastructure réseau joue un rôle essentiel dans la connectivité et la sécurité du système. Un réseau sécurisé, basé sur des protocoles HTTPS et des mécanismes d'authentification comme Sanctum, assure une communication fiable et protégée entre tous les composants, qu'ils soient locaux ou déployés dans le cloud. Cette infrastructure garantit l'intégrité des données et la confidentialité des informations médicales tout au long du flux de traitement.

Connexions et protocoles Les interactions entre les composants s'appuient sur des protocoles standardisés pour assurer une communication fluide et sécurisée. Les interfaces web et mobile communiquent avec le serveur backend via une API REST basée sur HTTP/HTTPS, complétée par des WebSockets pour les mises à jour en temps réel. Les flux vidéo provenant des caméras IP sont traités localement ou dans le cloud, où YOLO et Tesseract OCR analysent les images pour extraire les données vitales. Les alertes critiques sont diffusées via Firebase Cloud Messaging (FCM), permettant une notification rapide et fiable du personnel médical.

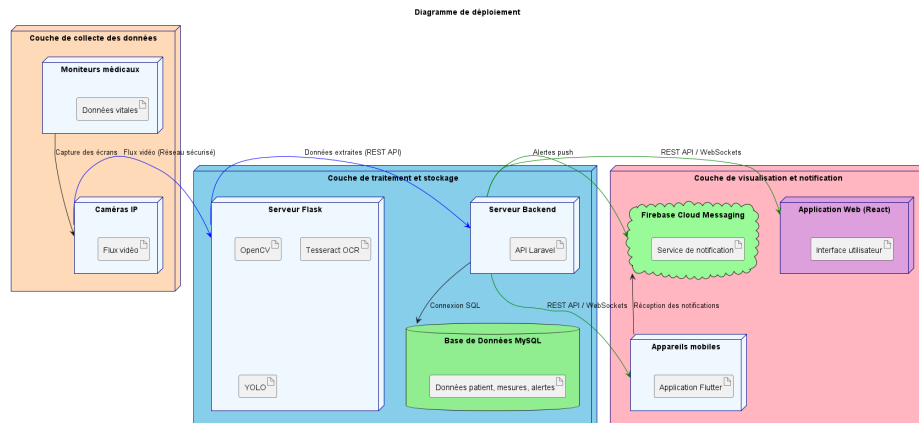


FIGURE 3.4 : Diagramme de déploiement

La Figure 3.4 illustre le déploiement des composants du système VitalSync à travers son architecture distribuée. Les moniteurs médicaux et les caméras IP, regroupés dans la couche de collecte, transmettent les images des signes vitaux au serveur central. Ce dernier, au sein de la couche de traitement et stockage, exécute les services Flask pour le traitement d'images (YOLO, OpenCV, Tesseract) et l'API Laravel pour la gestion des données, stockées dans une base MySQL. Les interfaces utilisateur, déployées sur des appareils mobiles (Flutter) et des navigateurs (React), accèdent aux données via une API REST et reçoivent des notifications push via FCM. Un réseau sécurisé relie tous les composants, assurant une communication rapide, fiable et protégée. Cette organisation permet une scalabilité et une robustesse adaptées aux besoins des environnements médicaux critiques.

3.2.4 Diagramme de composants

The backend of the system is designed to handle core functionalities through a combination of robust technologies. Python modules are utilized for advanced data processing, including the detection of data zones using the YOLO algorithm, optical character recognition (OCR) with Tesseract, and comprehensive data analysis. Additionally, Laravel services manage critical operations such as user authentication, alert management, and notification delivery, ensuring secure and efficient backend performance.

The frontend provides an intuitive and responsive user experience through two primary interfaces. React-based dashboards enable effective visualization of data, offering users clear and interactive insights into the processed information. Complementing this, a Flutter application delivers mobile alerts, allowing users to receive timely notifications and stay informed on the go.

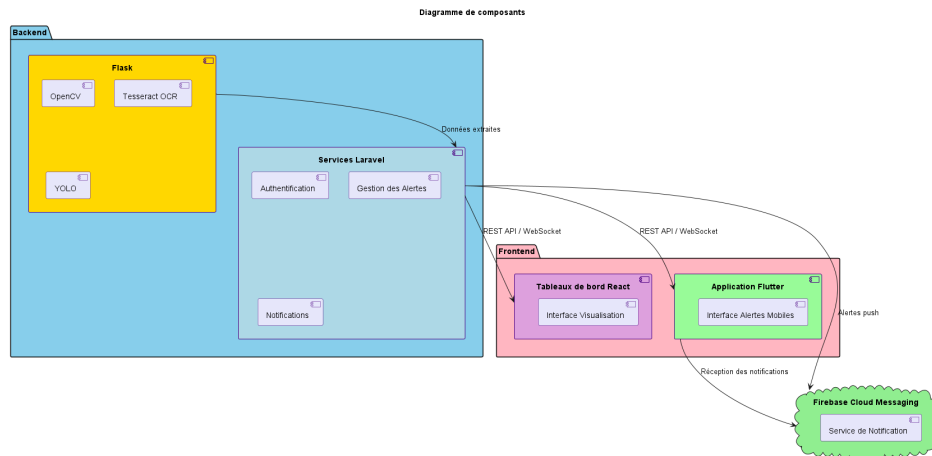


FIGURE 3.5 : Diagramme de composants du système

3.2.5 Diagramme d'activité

Le diagramme d'activité présenté dans la Figure 3.6 illustre les différentes étapes du traitement des données au sein du système VitalSync, depuis la collecte des données jusqu'à la notification des alertes. Il met en évidence les actions séquentielles réalisées par les différents modules, en soulignant les conditions qui déclenchent des alertes médicales. Cette représentation permet de visualiser clairement le parcours des données capturées par les caméras IP, traitées par les services intelligents (YOLO, OpenCV, Tesseract), puis analysées par le backend afin de détecter d'éventuelles anomalies. En cas de dépassement de seuils critiques, une alerte est générée et transmise en temps réel au personnel médical via des interfaces web et mobiles, garantissant ainsi une intervention rapide et efficace.

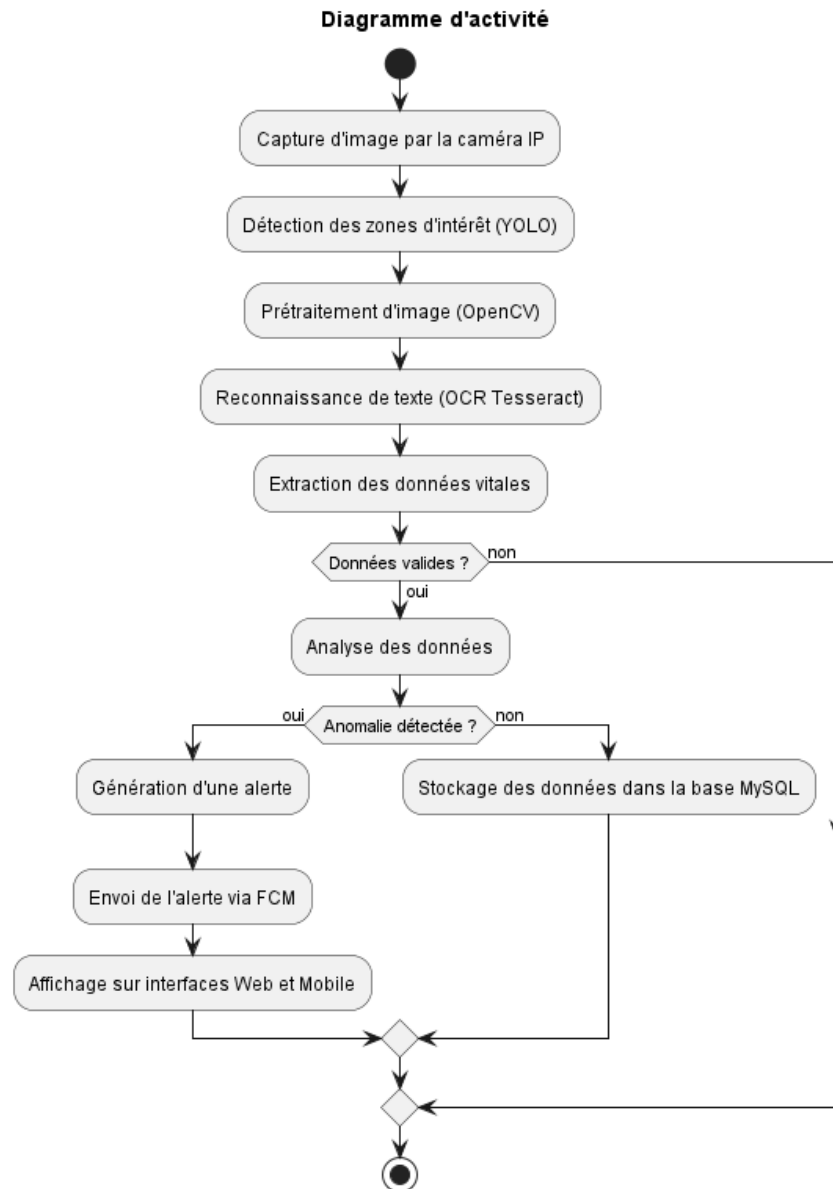


FIGURE 3.6 : Diagramme d'activité du flux de données

3.3 Architecture physique et fonctionnement

3.3.1 Composants matériels

Les composants matériels du système sont conçus pour faciliter une collecte, un traitement et une communication efficaces des données. Les moniteurs médicaux recueillent des données vitales telles que la fréquence cardiaque, la tension artérielle et la saturation en oxygène, affichant ces mesures sur leurs écrans. Les caméras capturent ces écrans et transmettent les images au service de traitement, où les technologies YOLO et OCR analysent les données.

Les serveurs, hébergés localement, stockent et traitent ces données en exécutant YOLO, OpenCV et Tesseract pour l'analyse d'images. Un réseau de communication sécurisé assure la transmission fiable des données entre les moniteurs, les caméras, les serveurs et les applications. Les appareils mobiles permettent au personnel médical de recevoir des notifications push en temps réel et d'accéder aux données vitales, améliorant ainsi la réactivité.

3.3.2 Fonctionnement

Le système fonctionne en trois étapes principales : collecte des données, traitement des données et génération d'alertes. Durant l'étape de collecte, les caméras capturent les écrans des moniteurs médicaux, et le modèle YOLO identifie les zones de données, telles que la fréquence cardiaque ou la SpO2. Ces images sont prétraitées avec OpenCV, et Tesseract OCR extrait les valeurs des signes vitaux.

Lors de l'étape de traitement, le système analyse les données en temps réel pour détecter les anomalies en les comparant à des seuils prédéfinis, stockant les résultats dans une base de données MySQL pour une analyse ultérieure. Enfin, lors de la génération d'alertes, celles-ci sont déclenchées lorsque les seuils sont dépassés et sont envoyées au personnel médical via FCM, tout en étant affichées sur les interfaces web et mobiles pour un accès immédiat.

3.3.3 Spécifications logicielles

L'architecture logicielle est composée de plusieurs éléments intégrés pour garantir une fonctionnalité robuste. Le backend utilise Laravel pour gérer l'authentification des utilisateurs, la manipulation des données et les services de notification via FCM, tandis que Python gère le traitement des données, intégrant YOLO pour la détection des zones de données, l'OCR avec OpenCV et Tesseract, ainsi que l'analyse d'images.

L'interface frontend comprend une interface web basée sur React pour la visualisation interactive des données et la gestion des alertes, complétée par une application mobile Flutter pour les notifications en temps réel et l'accès aux données. Une base de données MySQL stocke les données des patients, les alertes et les historiques. La communication est facilitée par des API REST pour les interactions entre le frontend, le backend et l'application mobile, avec WebSockets permettant des notifications en temps réel.

Parmi les outils supplémentaires figurent YOLOv8 intégré avec PyTorch pour la détection des zones de données, Docker pour la conteneurisation, Git pour le contrôle de version et Swagger pour le test des API REST.

3.3.4 Flux de communication

Le flux de communication commence par la collecte des données depuis les moniteurs médicaux, où YOLO détecte les zones de données et l'OCR extrait les informations pertinentes. Les données sont transmises de manière sécurisée au backend via un réseau protégé. Le backend traite les données en temps réel, les stocke dans une base de données MySQL et génère des alertes si des anomalies sont détectées.

Ces alertes sont envoyées au personnel médical via FCM. Les utilisateurs peuvent accéder aux données et aux alertes via l'application web ou mobile, assurant une communication fluide et rapide à travers le système.

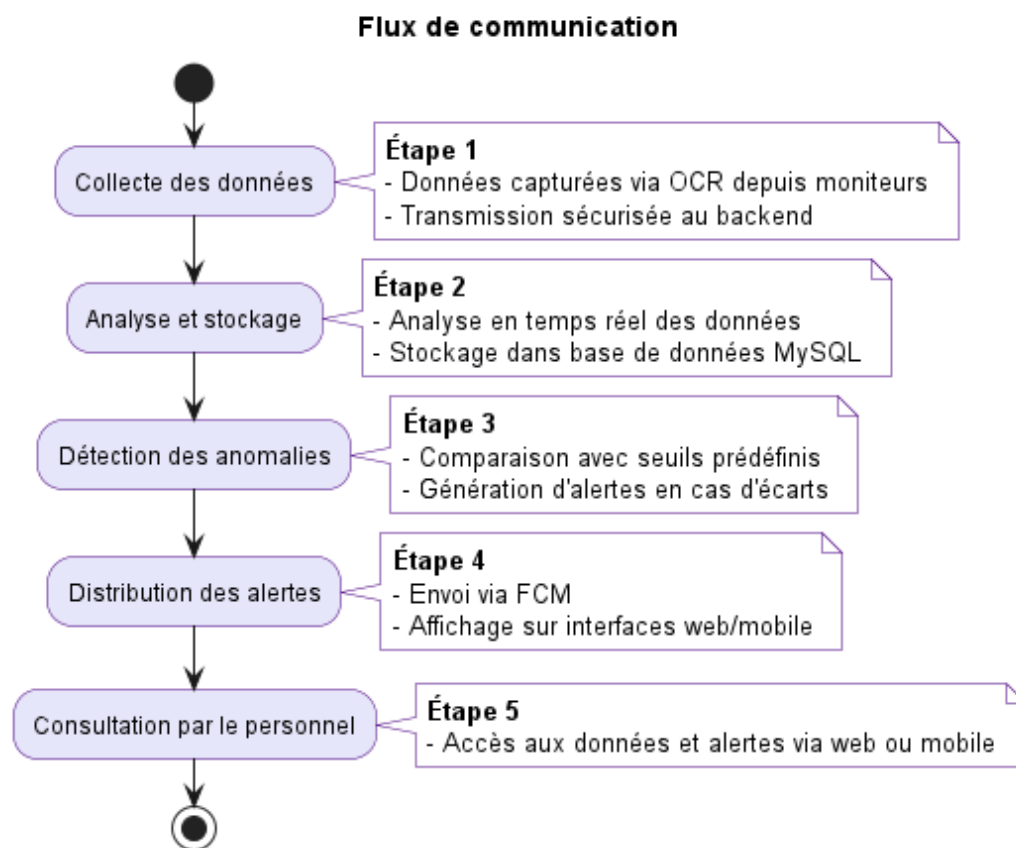


FIGURE 3.7 : Schéma général du flux de communication

3.4 Justification de l'architecture

L'architecture proposée garantit plusieurs qualités essentielles. Sa conception modulaire sépare le système en couches distinctes — collecte avec YOLO, traitement et notification — facilitant la maintenance et l'évolutivité. Le système assure une grande réactivité en fournissant des

alertes en temps réel, soutenu par une détection précise des zones de données via YOLO, permettant des réponses rapides aux situations critiques.

L'interopérabilité est maintenue grâce à la compatibilité avec YOLO, l'OCR et les standards d'échange de données médicales, assurant une intégration transparente avec les systèmes existants. De plus, l'architecture offre une flexibilité, permettant un déploiement sur des serveurs locaux ou cloud pour répondre aux besoins variés des hôpitaux.

3.5 Conclusion

L'architecture proposée, renforcée par l'intégration de YOLO pour une détection précise des zones de données, garantit une collecte fiable des données, un traitement efficace et une diffusion rapide des alertes tout en respectant les contraintes de sécurité et d'interopérabilité. Sa conception modulaire permet des améliorations futures, telles que l'intégration de nouveaux dispositifs ou l'ajout de fonctionnalités analytiques avancées, en faisant une solution robuste et adaptable pour la surveillance médicale.

Chapitre 4

Sprint 0

4.1 Introduction

Le Sprint 0 constitue la phase initiale du projet, qui vise à développer un système de collecte de données en temps réel et de gestion des alertes pour les appareils médicaux en salle de réveil. Cette étape fondamentale permet d'identifier les acteurs clés, de recueillir les besoins fonctionnels et non fonctionnels, et de définir la méthodologie Scrum qui guidera les phases ultérieures du développement. L'objectif est d'assurer un alignement avec les ambitions du projet, à savoir améliorer la surveillance post-opératoire grâce à une collecte automatisée des données et à des alertes en temps réel, comme décrit dans les chapitres d'introduction générale et d'étude architecturale.

4.2 Capture des besoins

4.2.1 Identification des acteurs

Le système interagit avec plusieurs acteurs essentiels, chacun jouant un rôle distinct dans son fonctionnement. Ces acteurs, résumés dans un tableau dédié, sont en phase avec le contexte médical et les composants architecturaux décrits dans les chapitres précédents. Le médecin consulte les données des patients, reçoit les alertes critiques et valide les actions nécessaires. L'infirmier ou l'infirmière surveille les patients, reçoit des notifications adaptées à ses horaires et agit immédiatement en cas de besoin. L'administrateur du système gère les utilisateurs, configure les seuils d'alerte et veille au bon fonctionnement de l'ensemble. Les moniteurs de signes vitaux fournissent les données vitales, transmises via une interface API ou par extraction optique à l'aide de caméras. Enfin, les caméras capturent les écrans des moniteurs pour permettre l'extraction des données grâce à des technologies de détection et de reconnaissance de texte.

TABLE 4.1 : Acteurs du système et leurs rôles

Acteur	Description
Médecin	Consulte les données des patients, reçoit les alertes critiques et valide les actions nécessaires.
Infirmier(ère)	Surveille les patients, reçoit les alertes en fonction de ses horaires et prend des mesures immédiates.
Administrateur du Système	Gère les utilisateurs, configure les seuils d'alerte et assure le bon fonctionnement du système.
Moniteur de Signes Vitaux	Source de données des signes vitaux, transmises via API ou OCR basé sur caméra.
Caméra	Capture les écrans des moniteurs pour l'extraction des données via YOLO et OCR.

4.2.2 Besoins fonctionnels et non fonctionnels

Les besoins fonctionnels et non fonctionnels traduisent les objectifs du système, qui consistent à automatiser la collecte des données, optimiser la gestion des alertes, et garantir l'évolutivité ainsi que la conformité aux normes médicales, comme mentionné dans le chapitre d'introduction.

En termes de fonctionnalités, le système collecte en temps réel les données issues des moniteurs de signes vitaux, tels que ceux des fabricants courants, en utilisant des caméras associées à une technologie de reconnaissance optique de caractères (OCR). Il extrait les signes vitaux, comme la fréquence cardiaque, la pression artérielle ou la saturation en oxygène, grâce à des outils de détection avancés et d'OCR, permettant une reconnaissance précise des valeurs affichées. Le système génère et gère des alertes en fonction de seuils prédéfinis, ajustables dynamiquement selon les besoins. Les données et les alertes sont accessibles via des interfaces web et mobiles, conçues pour offrir une visualisation en temps réel et une gestion intuitive. L'authentification et la gestion des utilisateurs s'appuient sur un système sécurisé basé sur les rôles, garantissant un accès adapté aux médecins, infirmiers et administrateurs. Les notifications sont envoyées en temps réel par des canaux variés, tels que des notifications push. Enfin, les données et l'historique des alertes sont stockés dans une base de données relationnelle pour permettre un suivi et une analyse ultérieure.

En ce qui concerne les besoins non fonctionnels, le système doit être disponible en continu avec un temps d'arrêt minimal, garantissant une accessibilité constante pour les utilisateurs. La sécurité des données est assurée par un chiffrement robuste et des mécanismes d'authentification fiables, protégeant ainsi les informations sensibles des patients. Le système est conçu pour être compatible avec une variété de moniteurs médicaux, assurant une interopérabilité efficace. Il est également capable de s'adapter à une augmentation du nombre de patients et d'appareils, garantissant une évolutivité optimale. Les alertes critiques sont traitées instantanément pour répondre aux situations urgentes, et le système respecte les réglementations en vigueur concernant la protection des données médicales.

4.2.3 Diagramme des cas d'utilisation global

Le diagramme global des cas d'utilisation illustre les interactions entre les acteurs et le système, incluant la consultation des données, la réception des alertes et la gestion administrative. Des prototypes d'interfaces web et mobiles ont été conçus pour visualiser les données et gérer les alertes, en tenant compte des besoins des utilisateurs identifiés précédemment.

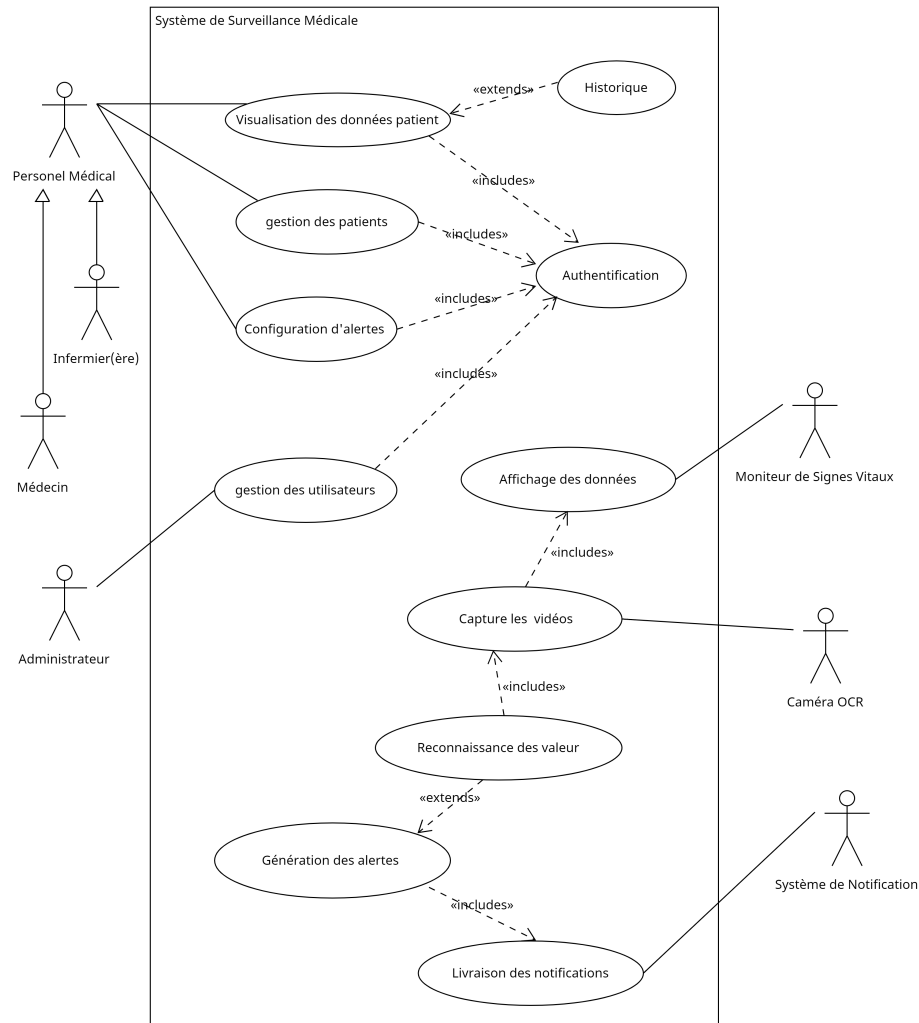


FIGURE 4.1 : Diagramme des cas d'utilisation global

4.3 Pilotage du projet avec Scrum

Le projet adopte la méthodologie Scrum pour assurer un développement itératif et une collaboration étroite avec les parties prenantes, comme justifié dans le chapitre consacré à la méthodologie. Cette section détaille les rôles Scrum, l'organisation du backlog et la planification des sprints pour le Sprint 0.

4.3.1 Acteurs Scrum et Missions

Les rôles Scrum et leurs responsabilités sont définis dans un tableau spécifique, garantissant une approche collaborative alignée sur les principes décrits précédemment. Le Product Owner est chargé de définir et de prioriser les fonctionnalités du produit, tout en validant les livrables pour s'assurer qu'ils répondent aux besoins du projet. L'équipe de développement conçoit, développe et teste les fonctionnalités, en veillant à la qualité du code produit. Le Scrum Master facilite les événements Scrum, élimine les obstacles et veille à ce que l'équipe adhère aux pratiques de la méthodologie.

TABLE 4.2 : Acteurs du projet Scrum

Rôle	Mission	Acteur
Product Owner	Définir et prioriser les fonctionnalités du produit (Product Backlog), valider les livrables.	Pr Ammeri Charfeddine
Équipe de Développement	Concevoir, développer et tester les fonctionnalités, assurer la qualité du code.	Zaibi Racha
Scrum Master	Faciliter les événements Scrum, supprimer les obstacles, assurer l'adhésion à Scrum.	Ouni Ghada

4.3.2 Organisation du Backlog

Le Product Backlog est géré à l'aide d'un outil de gestion de projet, organisé selon la méthode de priorisation MoSCoW pour refléter les objectifs stratégiques du projet. Chaque fonctionnalité est formulée sous forme de User Story pour garantir un développement centré sur l'utilisateur. Les tâches prioritaires incluent la détection et la reconnaissance des valeurs vitales à l'aide de technologies de détection et d'OCR, la génération et la gestion des alertes basées sur des seuils prédéfinis, ainsi que l'envoi de notifications via des canaux multiples, tels que les notifications push et les e-mails. Les interfaces web et mobiles permettent la visualisation des données et la gestion des alertes, tandis que l'authentification et la gestion des utilisateurs assurent un accès sécurisé basé sur les rôles. Le stockage des données et des historiques dans une base de données relationnelle est également prioritaire, tout comme la conformité aux réglementations médicales en matière de sécurité des données. D'autres tâches, jugées importantes mais non critiques pour la première version, incluent la configuration personnalisée des seuils d'alerte par patient, la gestion des horaires pour le filtrage des alertes, et l'optimisation des notifications pour une faible latence. Des fonctionnalités optionnelles, comme le support multilingue ou la gestion des alertes par niveau de priorité, pourraient être

ajoutées si les ressources le permettent. Enfin, certaines tâches, telles que l'analyse avancée des données avec intelligence artificielle, sont hors du périmètre actuel.

TABLE 4.3 : Backlog du Projet avec Priorités MoSCoW

ID	Tâche	Priorité
M-01	Détection et reconnaissance des valeurs vitales via YOLO et OCR pour les moniteurs.	M
M-02	Génération et gestion des alertes en fonction des seuils prédéfinis.	M
M-03	Envoi d'alertes via notifications push et e-mail.	M
M-04	Interface web et mobile pour visualiser les données et gérer les alertes.	M
M-05	Authentification et gestion des utilisateurs (médecins, infirmiers, administrateurs).	M
M-06	Stockage et historique des données dans une base de données.	M
M-07	Conformité aux réglementations médicales (sécurité des données).	M
S-02	Configuration des seuils d'alerte personnalisés par patient.	S
S-03	Gestion des horaires de filtrage des alertes pour le personnel médical.	S
S-04	Interface d'administration pour configurer les seuils et gérer les utilisateurs.	S
S-05	Optimisation des notifications push pour faible latence.	S
C-04	Support multilingue pour l'interface utilisateur.	C
C-05	Gestion des alertes par priorité (critique, moyen, faible).	C
C-06	Historique des alertes avec filtrage par date, patient, ou type d'alerte.	C
W-03	Analyse avancée des données avec intelligence artificielle pour le diagnostic.	W

4.3.3 Planification des sprints

Le projet est structuré en trois sprints, chacun d'une durée de trois semaines, comme illustré dans le chapitre consacré à la méthodologie Scrum. La réunion de planification du Sprint 0 a permis d'identifier les tâches et les livrables, en s'alignant sur les besoins des parties prenantes et les composants architecturaux décrits précédemment.

Le premier sprint se concentre sur la création d'un dataset et la mise en place d'un pipeline OCR. L'objectif est de développer un système capable d'extraire les données des moniteurs médicaux à l'aide de technologies de détection et d'OCR. Les livrables incluent un dataset

annoté d'images provenant de différents moniteurs, un modèle entraîné pour détecter les zones affichant les signes vitaux, et un pipeline intégrant la détection et l'extraction de texte. Les tâches principales consistent à capturer et annoter les images des moniteurs, entraîner le modèle de détection, intégrer des outils de traitement d'image et d'OCR, et configurer les caméras pour la capture des données. Les critères d'acceptation garantissent une détection précise des zones d'intérêt, une extraction fiable des valeurs, une faible latence du pipeline, et une compatibilité avec différents modèles de moniteurs. Les risques incluent des problèmes liés à l'éclairage ou à la qualité des images, qui peuvent être mitigés par un prétraitement optimisé, ainsi qu'une taille insuffisante du dataset, compensée par des techniques d'augmentation des données.

Le deuxième sprint se focalise sur la mise en place d'un système d'alertes en temps réel et de notifications adaptées aux rôles des utilisateurs. Les livrables comprennent un moteur d'alertes basé sur des règles définissant les seuils pour les signes vitaux, un système de notification différencié selon les rôles, et l'intégration de notifications push pour les appareils mobiles. Les tâches incluent la configuration des seuils d'alerte, l'association des notifications aux rôles appropriés, l'intégration d'un service de messagerie pour les notifications, et la validation des données extraites pour garantir leur fiabilité. Les critères d'acceptation vérifient que les alertes sont déclenchées rapidement, envoyées aux bons destinataires, et que les données sont validées avec une grande précision. Les risques incluent des retards dans la transmission des notifications, qui peuvent être atténués par des tests avec des données simulées, et des erreurs dans l'extraction des données, compensées par des tests unitaires rigoureux.

Le troisième sprint vise à développer les interfaces utilisateur et un tableau de bord pour la gestion des alertes. Les livrables incluent un tableau de bord web pour afficher les alertes et les signes vitaux en temps réel, une application mobile permettant la réception et l'accusé de réception des alertes, et une base de données pour stocker les alertes et les réponses du personnel. Les tâches consistent à développer le tableau de bord avec une priorisation claire des alertes, concevoir l'application mobile avec intégration des notifications, configurer la base de données pour un stockage fiable, et implémenter des technologies de mise à jour en temps réel. Les critères d'acceptation garantissent des mises à jour rapides des interfaces, une réception fiable des notifications, une intégrité totale des données stockées, et une ergonomie validée par les utilisateurs. Les risques incluent une latence potentielle des interfaces, mitigée par une optimisation des appels réseau, et une complexité d'intégration, résolue par des prototypes préliminaires et des tests itératifs.

4.4 Environnement de travail

4.4.1 Environnement matériel

L'environnement matériel est conçu pour répondre aux exigences architecturales décrites précédemment. Des caméras IP capturent les écrans des moniteurs médicaux pour permettre le traitement des données par détection et OCR. Des serveurs locaux dédiés assurent le stockage, le traitement des données et la génération des alertes. Un réseau local sécurisé avec accès Internet facilite la transmission des données entre les caméras, les serveurs et les applications. Enfin, des smartphones et tablettes compatibles permettent aux utilisateurs d'accéder aux notifications et aux données en temps réel.

4.4.2 Environnement de développement

L'environnement de développement s'appuie sur un ensemble cohérent de langages de programmation, frameworks et bibliothèques, choisis en fonction des exigences techniques du projet VitalSync. Chaque composant du système est développé avec des outils adaptés à sa fonction :

- **PHP avec Laravel** : utilisé pour développer l'API backend principale. Laravel offre une architecture MVC robuste, une gestion avancée de l'authentification (Sanctum), ainsi qu'un système de routage et de notifications efficace.



FIGURE 4.2 : Logos des technologies backend : Laravel et PHP

- **Python avec Flask** : utilisé pour les microservices de traitement d'image. Flask, léger et modulaire, est associé à des bibliothèques comme OpenCV (prétraitement d'image), YOLO (détection des zones d'intérêt) et Tesseract OCR (extraction de texte).



FIGURE 4.3 : Logos des technologies utilisées pour le traitement et l'API Python : Python et Flask

- **Dart avec Flutter** : utilisé pour l'application mobile multiplateforme. Flutter permet de développer des interfaces fluides et réactives, compatibles Android/iOS, et intègre Firebase Cloud Messaging (FCM) pour les notifications.



FIGURE 4.4 : Logos des technologies utilisées pour le développement mobile : Flutter et Dart

- **JavaScript avec React** : utilisé pour l'interface web destinée au personnel médical. React assure une expérience utilisateur moderne avec une gestion efficace de l'état des composants et des interactions dynamiques.



FIGURE 4.5 : Logos des technologies utilisées pour le développement web : React et JavaScript

- **MySQL avec phpMyAdmin** : MySQL est la base de données relationnelle choisie pour le stockage des patients, mesures vitales, et alertes. phpMyAdmin facilite la visualisation et la gestion de la base.



FIGURE 4.6 : Logos des outils de gestion de base de données : MySQL et phpMyAdmin

Ces technologies sont intégrées dans un environnement modulaire et conteneurisé à l'aide de Docker, garantissant une portabilité entre les environnements de développement, de test et de production. Le développement collaboratif est facilité par GitHub, tandis que l'édition de code est principalement réalisée avec Visual Studio Code et Android Studio.

4.4.3 Environnement logiciel

L'environnement logiciel repose sur des systèmes d'exploitation adaptés pour le développement et les appareils mobiles. Une base de données relationnelle garantit un stockage fiable et conforme aux standards. Les communications s'appuient sur des protocoles sécurisés, tels que des API REST avec authentification par tokens et des WebSockets pour les mises à jour en temps réel. Les outils de développement suivants sont employés :

- **Visual Studio Code** : Utilisé comme environnement de développement intégré, Visual Studio Code permet de coder, tester et déboguer efficacement grâce à son interface intuitive et ses extensions pour divers langages et frameworks.

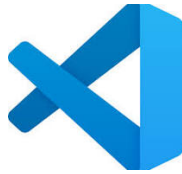


FIGURE 4.7 : Logo de Visual Studio Code

- **Android Studio** : Android Studio est utilisé pour le développement et le test des applications mobiles, offrant des outils spécialisés pour la création d'interfaces utilisateur et la gestion des performances sur les appareils Android.



FIGURE 4.8 : Logo d'Android Studio

- **LaTeX** : LaTeX est utilisé pour la rédaction du rapport final, offrant une mise en forme professionnelle et structurée. Il permet de générer des documents de haute qualité avec une gestion précise des sections, des références et des figures.



FIGURE 4.9 : Logo de LaTeX

- **GitHub** : GitHub facilite la gestion des versions et la collaboration sur le code, assurant un suivi rigoureux des modifications et une intégration fluide avec d'autres outils de développement.



FIGURE 4.10 : Logo de GitHub

- **Docker Desktop** : Docker Desktop permet de conteneuriser les applications, assurant une portabilité et une cohérence des environnements logiciels à travers les différentes phases du projet.



FIGURE 4.11 : Logo de Docker Desktop

4.5 Conclusion

Le Sprint 0 pose une base solide pour le projet en définissant les acteurs, les besoins et le cadre Scrum. Le Product Backlog priorisé, la planification détaillée des sprints et l'environnement de développement spécifié garantissent un alignement avec les objectifs de collecte de données en temps réel et de gestion des alertes. L'intégration de technologies de détection et d'OCR positionne le système pour un développement efficace dans les sprints suivants, tout en restant fidèle à l'architecture décrite dans les chapitres précédents.

Chapitre 5

Sprint 1 : Création du dataset et mise en place du pipeline OCR

5.1 Introduction

Ce chapitre décrit le Sprint 1, première itération du développement d'un système de collecte de données en temps réel et de gestion des alertes pour les appareils médicaux en salle de réveil, comme présenté dans les chapitres d'introduction générale et d'étude architecturale. Selon la planification établie dans le chapitre précédent, cette phase se concentre sur la création d'un dataset annoté à partir de moniteurs médicaux et sur le développement d'un pipeline OCR basé sur des technologies de détection et de reconnaissance de texte pour extraire les signes vitaux, tels que la fréquence cardiaque, la pression artérielle et la saturation en oxygène. Cette itération pose les bases techniques pour la collecte automatisée des données, un objectif central du projet, en suivant la méthodologie Scrum décrite dans le chapitre consacré à la méthode.

5.2 Objectifs du Sprint 1

L'objectif principal du Sprint 1 est de développer un pipeline fonctionnel permettant la collecte en temps réel des données affichées sur les écrans des moniteurs à l'aide de caméras IP. Cette phase vise à créer un dataset annoté d'images des moniteurs, avec des annotations précises pour identifier les signes vitaux. Un modèle de détection est entraîné pour repérer les régions d'intérêt contenant ces données. Des outils de traitement d'image et de reconnaissance optique de caractères (OCR) sont intégrés pour extraire les valeurs textuelles de ces régions. Les caméras IP sont configurées pour garantir une capture stable et optimale des images. Enfin, une API REST est développée pour permettre l'intégration du pipeline dans les phases suivantes. Ces objectifs s'alignent sur les tâches prioritaires du Product Backlog et les besoins fonctionnels d'interopérabilité avec différents moniteurs, comme détaillé dans le chapitre précédent.

5.2.1 Sprint Backlog

Le Sprint Backlog pour le Sprint 1 regroupe les tâches sélectionnées du Product Backlog (voir Tableau 4.3) pour répondre aux objectifs de cette itération. Chaque tâche est associée à une priorité, une estimation de l'effort en heures, et des critères d'acceptation pour garantir la qualité des livrables. Les tâches incluent la configuration du matériel, la capture et l'annotation des images, l'entraînement du modèle de détection, et le développement du pipeline OCR avec une API REST.

TABLE 5.1 : Sprint Backlog du Sprint 1

ID	Tâche	Priorité	Effort (h)	Critères d'Acceptation
M-01	Détection et reconnaissance des valeurs vitales via YOLO et OCR pour les moniteurs.	M	80	Précision de détection > 90%, OCR extrait les valeurs avec < 5% d'erreurs, compatible avec 3 moniteurs différents.
M-06	Stockage et historique des données dans une base de données.	M	20	Base de données relationnelle configurée, stockage des données extraites testé avec succès.
S-04	Interface d'administration pour configurer les seuils et gérer les utilisateurs.	S	30	Interface de base fonctionnelle pour configurer les paramètres de capture et d'OCR.

5.3 Tâches réalisées

Les tâches réalisées reflètent les engagements pris lors de la planification du Sprint 1 et sont organisées en sous-sections pour plus de clarté.

5.3.1 Configuration du matériel

L’environnement matériel a été préparé pour répondre aux besoins du pipeline. Des caméras IP ont été configurées à l’aide d’une application mobile dédiée, offrant une interface intuitive pour gérer les paramètres de capture. Un serveur local a été mis en place pour supporter l’entraînement du modèle de détection et l’exécution du pipeline OCR. Ce serveur utilise un système d’exploitation adapté, avec une accélération matérielle pour optimiser les performances. Les bibliothèques nécessaires au traitement des images et à l’extraction de texte ont été installées via un gestionnaire de dépendances.

5.3.2 Capture et annotation des images

La création du dataset a débuté par la capture d’images des écrans des moniteurs médicaux à l’aide de caméras IP. Ces images, prises dans des conditions d’éclairage contrôlé pour minimiser les reflets, ont été annotées à l’aide d’une plateforme spécialisée, en traçant des régions d’intérêt autour des paramètres vitaux, tels que la fréquence cardiaque, la pression artérielle et la saturation en oxygène. Le dataset a été divisé en ensembles pour l’entraînement, la validation et les tests, suivant une répartition standard. Pour améliorer la robustesse du modèle, des techniques d’augmentation des données, comme la rotation, le recadrage et l’ajustement de la luminosité, ont été appliquées, augmentant ainsi la taille du dataset d’entraînement.

5.3.3 Entraînement du modèle de détection

Un modèle de détection, choisi pour son équilibre entre précision et rapidité, a été entraîné sur la plateforme sélectionnée. Les paramètres d’entraînement ont été ajustés pour optimiser les performances, et le modèle a atteint une précision satisfaisante sur l’ensemble de validation, dépassant les critères d’acceptation établis. Des techniques comme l’ajustement du seuil de confiance et la suppression des détections redondantes ont été appliquées pour réduire les erreurs. Les résultats par catégorie de signes vitaux sont présentés dans un tableau dédié.

TABLE 5.2 : Précision par classe pour le modèle de détection sur le jeu de validation

Classe	Précision
Fréquence cardiaque	Bonne
Pression systolique	Très bonne
Pression diastolique	Très bonne
Saturation en oxygène	Très bonne
Pouls	Satisfaisante
Autres paramètres vitaux	Variable

FIGURE 5.1 : Courbe d'apprentissage du modèle de détection

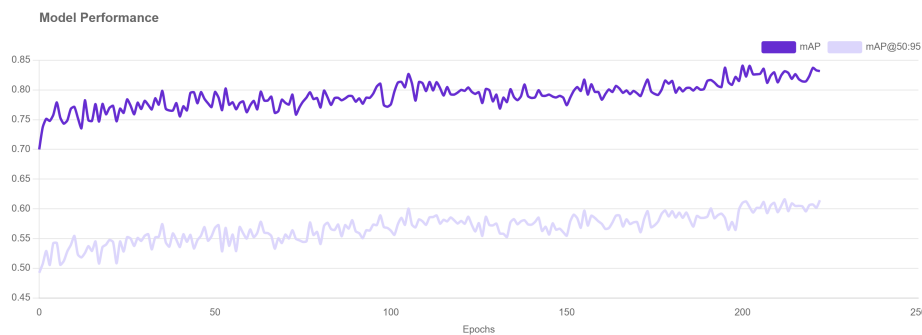


FIGURE 5.2 : Courbe d'apprentissage du modèle de détection

5.3.4 Développement du pipeline OCR

Le pipeline OCR a été conçu pour traiter les images capturées et extraire les données textuelles. Les images ont d'abord été prétraitées avec des outils de traitement d'image pour convertir les images en niveaux de gris, améliorer le contraste et réduire le bruit tout en préservant les contours. Les régions d'intérêt détectées ont été recadrées avec une marge suffisante pour inclure le texte complet. Une technologie OCR a été configurée avec des paramètres optimisés pour extraire les valeurs numériques, et des expressions régulières ont été utilisées pour valider les formats des données extraites, rejetant les sorties non conformes. Le pipeline a été implémenté dans un langage de programmation adapté, avec un framework web pour exposer une API REST. Cette API accepte des flux vidéo ou des images individuelles et renvoie les valeurs extraites dans un format structuré. La documentation de l'API a été générée pour faciliter son utilisation future. Les performances du pipeline, en termes de précision et de latence, ont respecté les critères d'acceptation.

5.3.5 Tests d'intégration et déploiement initial

Le pipeline a été testé en conditions réelles en connectant les caméras IP au serveur via un réseau sécurisé. Le flux vidéo a été capturé en continu, et les images ont été traitées à une fréquence adaptée pour simuler la collecte en temps réel. Le pipeline a été conteneurisé pour simplifier son déploiement sur le serveur, intégrant toutes les dépendances nécessaires. L'intégration a été validée sur plusieurs scénarios avec différents moniteurs et conditions d'éclairage, confirmant une compatibilité robuste.

5.4 Livrables du Sprint 1

Les livrables produits sont alignés sur les objectifs du Sprint 1. Un dataset annoté, comprenant des images des moniteurs pour l’entraînement, la validation et les tests, a été créé et stocké dans un dépôt avec des métadonnées. Un modèle de détection entraîné, sauvegardé dans un format standard, a été déployé sur le serveur. Le pipeline OCR, intégrant les technologies de détection, de traitement d’image et d’extraction de texte, expose une API REST avec une précision et une latence conformes aux attentes. Les caméras IP ont été configurées pour un flux vidéo stable, testées pour leur compatibilité avec différents moniteurs. Enfin, une documentation complète, incluant celle de l’API, un guide d’installation du pipeline et un rapport sur l’entraînement du modèle, a été produite.

5.5 Tests et validation

Les tests ont été conçus pour valider les critères d’acceptation définis dans le chapitre précédent. Le modèle de détection a atteint une précision satisfaisante, dépassant les attentes, avec des erreurs mineures corrigées par un prétraitement renforcé. L’OCR a extrait les données avec une précision élevée, respectant les critères, malgré une erreur isolée liée à une police floue, résolue par un ajustement du contraste. La latence du pipeline a été conforme aux exigences, et la compatibilité avec différents moniteurs a été validée dans tous les scénarios testés. La stabilité du flux vidéo a également été confirmée, avec un taux de perte de trames négligeable.

TABLE 5.3 : Résultats des tests du Sprint 1

Critère	Résultat	Statut
Précision du modèle de détection	Très bonne	Validé
Précision de l’OCR	Très bonne	Valé avec validation
Latence du pipeline	Conforme	Validé
Compatibilité avec moniteurs	Complète	Validé
Stabilité du flux vidéo	Excellente	Validé

5.6 Défis et solutions

Plusieurs défis ont été rencontrés lors du Sprint 1. Les reflets et variations d’éclairage sur les écrans des moniteurs ont affecté la détection et l’extraction des données. Ces problèmes ont été résolus par un réglage de l’exposition des caméras, l’application de filtres de prétraitement

et un ajustement physique des angles de capture. La taille initiale du dataset était limitée, ce qui aurait pu compromettre la robustesse du modèle. Une augmentation des données a permis de compenser ce manque, et des captures supplémentaires sont prévues pour le sprint suivant. Certaines polices floues ou petites ont posé des difficultés à l'OCR, mais des optimisations des paramètres et une validation rigoureuse des sorties ont permis de surmonter ce problème. Enfin, des contraintes d'accès aux ressources matérielles ont causé des retards, résolus par la mise en place d'un calendrier partagé pour les sprints futurs.

5.7 Rétrospective du Sprint 1

La rétrospective Scrum a permis d'évaluer les performances de l'équipe et d'identifier des axes d'amélioration. Le pipeline OCR a atteint les objectifs fixés, validant la faisabilité technique du projet. La collaboration a été efficace grâce à l'utilisation d'un outil de gestion de projet et à des réunions quotidiennes facilitées par le Scrum Master. La communication via une plateforme dédiée a permis de résoudre rapidement les problèmes techniques. Cependant, le temps d'annotation des images a été sous-estimé, nécessitant une planification plus réaliste pour le prochain sprint. Des tests précoces sur des conditions d'éclairage extrêmes auraient pu être anticipés, et une action corrective consistera à intégrer ces tests dès le début du Sprint 2. Les contraintes d'accès aux ressources matérielles et les difficultés initiales de configuration des caméras ont également été notées, avec des mesures prévues pour documenter les paramètres et réserver les ressources à l'avance.

5.8 Planification pour le Sprint 2

Le Sprint 1 a jeté les bases pour le Sprint 2, qui se concentre sur la validation des données extraites et la mise en place du système d'alertes. Les tâches préparatoires incluent l'intégration du pipeline OCR avec un module de validation pour garantir une précision élevée, la configuration des seuils d'alerte pour les signes vitaux, et l'intégration d'un service de notifications push basé sur l'API REST développée.

5.9 Conclusion

Le Sprint 1 a établi une fondation technique pour la collecte automatisée des données en temps réel, grâce à un format pipeline performant et un dataset annoté adapté aux moniteurs

médicaux. Les livrables respectent les critères d'acceptation en termes de précision, de latence et de compatibilité, et les défis rencontrés, tels que les reflets, les polices floues ou les contraintes matérielles, ont été résolus par des optimisations techniques et organisationnelles. La rétrospective a permis d'identifier des améliorations pour les sprints futurs, notamment en matière de planification et de tests précoces. Ce sprint prépare efficacement le terrain pour le Sprint 2, qui développera le système d'alertes et de notifications, renforçant l'alignement avec les objectifs du projet décrits dans le chapitre d'introduction.

Chapitre 6

Sprint 2

6.1 Introduction

Le Sprint 2 s'appuie sur le pipeline de traitement des données établi lors du Sprint 1, en mettant l'accent sur le développement des fonctionnalités de génération d'alertes, de livraison de notifications et d'interaction avec le personnel médical. Ces éléments sont cruciaux pour réaliser l'objectif du système VitalSync, qui vise à surveiller les signes vitaux en temps réel et à faciliter une réponse médicale rapide, conformément au flux de travail décrit dans le chapitre sur l'architecture. Ce chapitre présente les objectifs, les activités réalisées, les résultats obtenus, les tests effectués, les défis rencontrés, les réflexions issues de la rétrospective, ainsi que la planification pour le Sprint 3.

6.2 Objectifs

Le Sprint 2 visait à développer plusieurs fonctionnalités clés pour renforcer le système VitalSync. L'équipe s'est concentrée sur la mise en place d'un mécanisme d'évaluation des règles d'alerte permettant de détecter les anomalies dans les signes vitaux en fonction de seuils personnalisés pour chaque patient. En cas de dépassement de ces seuils, le système devait générer et enregistrer des alertes dans la base de données. Par ailleurs, des notifications étaient envoyées au personnel médical pour les alertes critiques via Firebase Cloud Messaging (FCM). Un tableau de bord initial, basé sur Laravel et React, a été conçu pour permettre au personnel médical de visualiser les alertes, consulter les données des patients et confirmer la réception des notifications. Enfin, l'API Flask a été enrichie pour intégrer la logique des alertes et des notifications, posant les bases d'une interface utilisateur améliorée.

6.3 Activités

Le Sprint 2, d'une durée de trois semaines, a été structuré autour de plusieurs activités clés, chacune contribuant à l'avancement du système.

6.3.1 Évaluation des règles d'alerte

Un système d'évaluation des alertes a été développé pour analyser les données des signes vitaux, telles que la fréquence cardiaque ou la pression artérielle, en les comparant aux seuils spécifiques définis pour chaque patient dans la base de données. L'API Flask a été étendue pour interroger ces règles et comparer les valeurs entrantes aux seuils minimum et maximum. Ce mécanisme a démontré une grande précision dans la détection des anomalies lors des tests initiaux.

6.3.2 Génération et stockage des alertes

Lorsqu'une anomalie était détectée, une alerte était générée et enregistrée dans la base de données, en lien avec les informations du patient et des données concernées. Le système prenait en charge différents niveaux de priorité et statuts pour les alertes, garantissant un suivi fiable sans perte de données dans les scénarios de test.

6.3.3 Livraison des notifications

Un système de notification basé sur Firebase Cloud Messaging a été mis en œuvre pour transmettre les alertes critiques au personnel médical. Les notifications étaient envoyées aux appareils des soignants identifiés dans la base de données, avec une latence moyenne conforme aux exigences de temps réel du système.

6.3.4 Confirmation des alertes

Un tableau de bord préliminaire, développé avec Laravel et React, permettait au personnel médical de visualiser les alertes et de confirmer leur réception. La confirmation mettait à jour le statut des alertes dans la base de données, assurant une traçabilité complète. Les préférences des utilisateurs étaient également enregistrées pour personnaliser l'expérience.

6.3.5 Intégration de l'API et du tableau de bord

L'API Flask a été optimisée pour gérer la logique des alertes et des notifications, tout en s'intégrant au backend Laravel. Le tableau de bord affichait les données des patients, les signes vitaux et les alertes, avec des fonctionnalités de filtrage de base, préparant le terrain pour des améliorations prévues au Sprint 3.

6.3.6 Mise en œuvre des alertes et notifications

Le flux de travail du Sprint 2, allant du traitement des données à la confirmation des alertes, est illustré dans un diagramme de séquence. Ce diagramme s'appuie sur le pipeline du Sprint 1 et intègre les étapes d'alerte et de notification décrites dans le flux global du système.

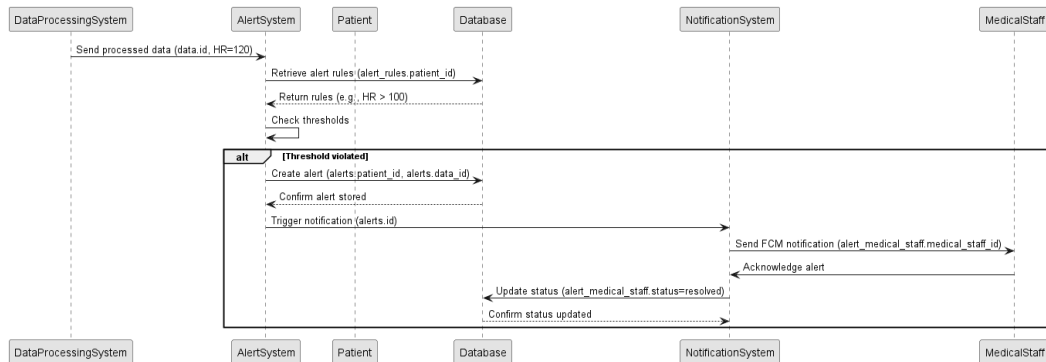


FIGURE 6.1 : Diagramme de séquence de la génération d'alertes et de la notification

6.4 Résultats

Le Sprint 2 a permis de livrer plusieurs composants essentiels. Un système d'alerte intégré à l'API Flask a été mis en place, capable de détecter les anomalies en fonction des seuils définis. Un système de notification utilisant FCM a assuré la transmission rapide des alertes au personnel médical. Un tableau de bord préliminaire Laravel/React a été développé pour la visualisation et la gestion des alertes. Les interactions avec la base de données ont été mises à jour pour inclure les tables nécessaires au suivi des alertes et des notifications. Enfin, une documentation détaillant les points d'API et l'utilisation du tableau de bord a été produite.

6.5 Tests

Plusieurs tests ont été réalisés pour valider les résultats du Sprint 2. La précision du système d'alerte a été évaluée à l'aide d'enregistrements simulés de signes vitaux, démontrant une détection fiable des anomalies. La latence des notifications a été mesurée sur un grand nombre d'alertes, confirmant que le système répondait aux exigences de temps réel avec un taux de succès élevé. Les fonctionnalités de confirmation des alertes ont été testées pour vérifier que les statuts étaient correctement mis à jour. Enfin, des tests d'intégration de bout en bout ont assuré que le flux de données, de la capture à la notification, fonctionnait sans erreur. Les résultats ont été consignés et examinés lors de la rétrospective.

6.6 Défis

Plusieurs obstacles ont été rencontrés lors du Sprint 2. La configuration initiale de FCM a posé des problèmes, entraînant des échecs occasionnels de livraison des notifications, qui ont été résolus en optimisant le middleware Laravel. La performance en temps réel a également été un défi, certaines alertes critiques présentant une latence légèrement supérieure aux attentes, ce qui a nécessité une optimisation des requêtes de la base de données. Enfin, les retours des utilisateurs ont souligné le besoin d'améliorer les fonctionnalités de filtrage du tableau de bord, une priorité reportée au Sprint 3.

6.7 Rétrospective

La rétrospective du Sprint 2 a permis d'identifier les points forts et les axes d'amélioration. L'équipe a réussi à livrer un système fonctionnel d'alertes et de notifications, atteignant une haute précision et une latence conforme aux attentes. Cependant, une meilleure estimation du temps nécessaire pour certaines tâches, comme l'intégration de FCM, aurait permis une planification plus précise. Les réunions quotidiennes ont été ajustées pour mieux se concentrer sur la résolution des obstacles. La collaboration entre les équipes backend et frontend s'est renforcée, mais les interactions avec le personnel médical pourraient être davantage optimisées pour accélérer les retours.

6.8 Planification du Sprint 3

Le Sprint 3 se concentrera sur l'amélioration des performances du système, notamment en réduisant davantage la latence des notifications. Le tableau de bord sera enrichi avec des fonctionnalités avancées de filtrage et des visualisations des données, telles que les tendances des signes vitaux. Des tests d'acceptation seront menés avec le personnel médical pour valider les fonctionnalités. Enfin, la documentation sera finalisée, et les préparatifs pour le déploiement seront amorcés, tout en respectant une durée de trois semaines et en priorisant les retours des utilisateurs et les métriques de performance.

6.9 Conclusion

Le Sprint 2 a marqué une avancée significative pour le système VitalSync en intégrant des fonctionnalités essentielles de gestion des alertes et des notifications. Les résultats obtenus, caractérisés par une haute précision et une faible latence, constituent une étape importante dans la réalisation du flux de travail global. Les défis liés à la configuration de FCM et à l'ergonomie du tableau de bord ont été surmontés, orientant les objectifs du Sprint 3 vers l'optimisation et la finalisation du système en vue de son déploiement.

Bibliographie

- [1] Faculté de Médecine de Monastir. Rapport annuel 2020, 2020. Available : <http://www.fmm.rnu.tn/>.
- [2] International Organization for Standardization. Iso 21001 :2018 - educational organizations, 2018. Available : <https://www.iso.org/standard/69596.html>.
- [3] World Health Organization. Digital health in low-resource settings, 2021. Available : <https://www.who.int/publications>.
- [4] A. Ben Abdelaziz et al. Challenges in tunisian public hospitals : A qualitative study. *La Tunisie Médicale*, 97(1) :45–52, 2019.
- [5] J. Smith et al. Impact of monitoring delays on post-operative outcomes. *Critical Care Medicine*, 48(3) :321–328, 2020.
- [6] World Health Organization. *Global Standards for the Initial Education of Professional Nurses and Midwives*. WHO Press, 2009.
- [7] Mindray. Patient monitoring systems, 2023. Available : <https://www.mindray.com/>.
- [8] United Nations. Technology and innovation report 2021, 2021. Available : <https://unctad.org/publication/technology-and-innovation-report-2021>.
- [9] F. Al-Turjman. Iot-enabled smart healthcare systems. *IEEE Access*, 8 :123456–123467, 2020.
- [10] R. Smith. An overview of the tesseract ocr engine. *International Conference on Document Analysis and Recognition (ICDAR)*, pages 629–633, 2007.
- [11] Google. Firebase cloud messaging, 2023. Available : <https://firebase.google.com/docs/cloud-messaging>.
- [12] World Health Organization. Medical devices : Managing the mismatch, 2010. Available : <https://www.who.int/publications>.
- [13] American Heart Association. *Advanced Cardiovascular Life Support Provider Manual*. AHA, 2020.
- [14] IQ Messenger. Smartapp features, 2023. Available : <https://www.iqmessenger.com/>.
- [15] L. Sun. Network reliability in healthcare systems. *Healthcare IT Review*, 12(4) :45–50, 2020.
- [16] Y. Zhang et al. Real-time iot healthcare monitoring. *Journal of Medical Systems*, 43(8) :1–10, 2019.
- [17] Ken Schwaber and Jeff Sutherland. *Scrum : The art of doing twice the work in half the time*. Crown Business, 2004.
- [18] Kenneth S. Rubin. *Essential Scrum : A practical guide to the most popular Agile process*. Addison-Wesley, 2012.

- [19] Atlassian. What is scrum? <https://www.atlassian.com/agile/scrum>, 2023.
- [20] Atlassian. Scrum artifacts. <https://www.atlassian.com/agile/scrum/artifacts>, 2023.
- [21] Atlassian. Scrum roles. <https://www.atlassian.com/agile/scrum/roles>, 2023.
- [22] Mike Cohn. *Succeeding with Agile : Software development using Scrum*. Addison-Wesley, 2009.
- [23] ClickUp. Clickup docs : Product management and agile tools. <https://docs.clickup.com/en/articles/2095287-getting-started-with-clickup>, 2025.
- [24] Slack Technologies. Slack documentation. <https://slack.com/help/categories/200111606>, 2025.
- [25] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.